

Pedro Henrique Martins de Santi

Redesign of a Micromouse Robot: Hardware, Software, and Control Integration for Intelligent Navigation

São Paulo, Brazil

2025

Pedro Henrique Martins de Santi

Redesign of a Micromouse Robot: Hardware, Software, and Control Integration for Intelligent Navigation

Redesign and optimization of an autonomous robot for the Micromouse category. This work encompasses the evolution of a control system, the implementation of advanced navigation algorithms, and the enhancement of the mechatronic platform, integrated to maximize speed and efficiency in maze exploration.

Universidade de São Paulo - USP

Escola Politécnica

Undergraduate Program

Supervisor: Bruno de Carvalho Albertini

São Paulo, Brazil

2025

Agradecimentos

Agradeço aos meus pais, Marcelo e Rita, e à minha namorada, Carolina, pelo suporte e apoio durante a toda a minha jornada acadêmica.

Agradeço aos membros do Time Micras: Matheus Rezende, Eduardo Barreto, João Retti e Cláudio Dardengo, pelo apoio na pesquisa e desenvolvimento.

Um agradecimento especial a Gabriel Cosme Barbosa. Sua colaboração foi fundamental nas discussões e definição dos requisitos do projeto, no desenvolvimento conjunto dos algoritmos de alto nível e na revisão dos projetos elétricos. Agradeço também pelo suporte logístico na manufatura dos componentes mecânicos e pelo apoio constante durante o projeto.

Ao professor Bruno Albertini, pela orientação e viabilização deste trabalho.

À equipe ThundeRatz de Robótica, que forneceu equipamentos, estrutura e componentes para o desenvolvimento do projeto, além de ter constituído minha base técnica na robótica.

À Escola Politécnica da Universidade de São Paulo, pela formação e base teórica.

Abstract

This monograph presents the complete redesign and optimization of an autonomous robotic platform for the Micromouse category. Motivated by the mechanical and computational limitations identified in the previous version, this iteration focuses on maximizing dynamic performance and system robustness. The project integrates a new differential drive mechanical architecture with a high-density embedded system, based on the STM32H7 microcontroller (550 MHz Cortex-M7) and a 4-layer PCB designed for signal integrity. The firmware, developed in C++ under a modular and decoupled architecture, introduces advanced algorithms for Local Odometry, queue-based trajectory planning, and Feed-Forward motion control. System validation was conducted through a hybrid approach, combining rigorous physical simulations with subsystem bench testing. The work culminates in the release of the entire hardware and software engineering as an open-source repository, aiming to foster technical development within the Brazilian competitive robotics community.

Keywords: Mobile Robotics. Micromouse. Embedded Systems. STM32H7. Path Planning. Feed-Forward Control.

Resumo

Esta monografia apresenta o reprojeto completo e a otimização de uma plataforma robótica autônoma para a categoria Micromouse. Motivada pelas limitações mecânicas e computacionais identificadas na versão anterior, esta iteração foca na maximização do desempenho dinâmico e na robustez do sistema. O projeto integra uma nova arquitetura mecânica de tração diferencial com um sistema embarcado de alta densidade, baseado no microcontrolador STM32H7 (Cortex-M7 de 550 MHz) e em uma PCB de 4 camadas projetada para integridade de sinal. O firmware, desenvolvido em C++ sob uma arquitetura modular e desacoplada, introduz algoritmos avançados de Odometria Local, planejamento de trajetória baseado em filas de ações e controle de movimento Feed-Forward. A validação do sistema foi realizada através de uma abordagem híbrida, combinando simulações físicas rigorosas com testes de subsistemas em bancada. O trabalho culmina na disponibilização de toda a engenharia de hardware e software como um repositório de código aberto, visando fomentar o desenvolvimento técnico da comunidade brasileira de robótica competitiva.

Palavras-chave: Robótica Móvel. Micromouse. Sistemas Embarcados. STM32H7. Planejamento de Trajetória. Controle Feed-Forward.

List of Figures

Figure 1 – Isometric view of the complete mechanical design.	39
Figure 2 – PCB CAD design.	39
Figure 3 – PCB detail showing countersunk holes for flush mounting.	39
Figure 4 – Right motor mount.	40
Figure 5 – Transmission set CAD.	41
Figure 6 – Fan Mount.	42
Figure 7 – Battery fixation system aiming for mass centralization.	42
Figure 8 – Part preparation in Lychee Slicer software.	43
Figure 9 – Parts freshly printed and after the cleaning/curing process.	43
Figure 10 – Fully assembled printed parts.	43
Figure 11 – PCB Altium ECAD project.	44
Figure 12 – Microcontroller circuit schematic.	45
Figure 13 – Power management and switching circuit PCB.	46
Figure 14 – Wall sensors schematics.	47
Figure 15 – Wall sensors on PCB.	47
Figure 16 – Encoder board PCB.	48
Figure 17 – Encoder board mating.	48
Figure 18 – Locomotion driver schematics.	49
Figure 19 – Locomotion driver PCB.	49
Figure 20 – Fan driver schematics.	50
Figure 21 – Fan driver PCB.	50
Figure 22 – Custom Bluetooth module schematics.	51
Figure 23 – Custom Bluetooth module PCB.	52
Figure 24 – PCB Stackup (Rotated Views).	53
Figure 25 – Main Board batch: PCBA (left) and bare PCBs (right).	54
Figure 26 – Auxiliary printed circuit boards designed for peripheral integration.	54
Figure 27 – Robot’s main FSM.	57
Figure 28 – Simulation environment: Robot executing the exploration algorithm and mapping the maze. The mapped walls are highlighted in blue, and the virtual walls in yellow	70
Figure 29 – Validation of the new software architecture running on the previous version hardware.	71
Figure 30 – Mechanical validation: 3D printed prototype assembly and PCB fit check.	72

List of Tables

Table 1 – Comparative analysis between the first and second versions of the robot.	73
--	----

Contents

1	INTRODUCTION	17
1.1	Motivation	17
1.2	Objective	18
1.3	Justification	18
1.4	Organization of the Work	19
2	CONCEPTUAL ASPECTS	21
2.1	Micro Mouse Category	21
2.1.1	The Robot: Physical and Functional Constraints	21
2.1.2	The Environment: Maze Specifications	21
2.1.3	Competition Dynamics: Exploration vs. Speed Run	22
2.2	Fundamentals of Mobile Robotics	22
2.2.1	Differential Kinematics	22
2.2.2	Odometry and Pose Estimation	23
2.2.2.1	The Challenge of Accumulative Error (Drift)	24
2.3	Control Theory and System Dynamics	24
2.3.1	Feedback Control Systems (PID)	24
2.3.2	Feed-Forward Control Architecture	25
2.3.3	Motion Profiling	25
2.3.4	Dynamics of Cornering	26
2.3.5	Aerodynamic Downforce Enhancement	26
2.4	Environment Representation and Search Algorithms	27
2.4.1	Graph-Based Representation	27
2.4.2	Breadth-First Search (BFS) and Flood Fill	27
2.4.2.1	Depth-First Search (DFS)	28
2.4.2.2	Optimization Techniques: Branch and Bound	28
3	METHODOLOGY	31
3.1	Development Planning	31
3.1.1	Critical Performance Analysis	31
3.1.2	Requirements Definition and Specification	31
3.1.3	High-Level Software Refactoring and Simulation	31
3.1.4	Mechatronic Co-Design	32
3.1.5	Manufacturing and Hybrid Assembly	32
3.1.6	System Integration and Validation Plan	32

4	REQUIREMENTS SPECIFICATIONS	33
4.1	Functional Requirements	33
4.2	Non-Functional Requirements	34
4.2.1	Mechanical Constraints	34
4.2.2	Electrical Constraints	34
4.2.3	Computational and Control Constraints	34
5	PROJECT DEVELOPMENT	35
5.1	Technologies Used	35
5.1.1	Mechanical Design and Manufacturing	35
5.1.1.1	SolidWorks	35
5.1.1.2	Anycubic Photon Mono 4 (10k)	35
5.1.1.3	Lychee Slicer	36
5.1.2	Electronic Design and Manufacturing	36
5.1.2.1	Altium Designer	36
5.1.2.2	JLCPCB (Manufacturing and Assembly)	36
5.1.3	Computation and Firmware	37
5.1.3.1	Visual Studio Code (VS Code)	37
5.1.3.2	Git & GitHub	37
5.1.3.3	STM32Cube Ecosystem	37
5.1.3.4	Debugging and Programming Toolchain	38
5.1.3.5	MuJoCo (Physics Engine)	38
5.2	Project and implementation	38
5.2.1	Mechanical Implementation	38
5.2.2	Structure and Integration (PCB-as-Chassis)	39
5.2.3	Drive System and Motor Mounts	40
5.2.4	Power Transmission	41
5.2.5	Suction System Support (Fan Mount)	41
5.2.6	Battery Fixation and Center of Mass	42
5.2.7	Manufacturing Process	43
5.2.8	Electrical Implementation	44
5.2.8.1	Microcontroller	44
5.2.8.2	Power Management Architecture	45
5.2.8.3	Sensing and Perception	46
5.2.8.4	Actuation and Drivers	49
5.2.8.5	Connectivity and Interface	51
5.2.8.6	PCB Design and Signal Integrity	52
5.2.8.7	Manufacturing and Assembly	53
5.3	Computational Implementation	55

5.3.1	Top Level Logic and Finite State Machine	56
5.3.1.1	Architectural Paradigm Shift	56
5.3.1.2	Finite State Machine (FSM)	56
5.3.1.3	State definitions	56
5.3.1.4	State Transitions and Flow	57
5.3.1.5	Human-Machine Interface (HMI) Abstraction	58
5.3.1.6	Event-Driven Configuration	58
5.3.1.7	Configuration Robustness	58
5.3.1.8	Communication and Telemetry System (micras_comm)	59
5.3.1.8.1	Packet Structure and Protocol	59
5.3.1.8.2	Serial Variable Pool	59
5.3.1.8.3	Service Execution Model	60
5.3.2	Navigation System Architecture	60
5.3.3	Localization and Mapping	61
5.3.3.1	Odometry	61
5.3.3.2	Costmap and Flood Fill	62
5.3.4	Exploration and Path Planning	62
5.3.4.1	Wall Detection and Updates	62
5.3.4.2	Exploration Strategy (BFS)	62
5.3.4.3	Dead End Optimization	62
5.3.4.4	Fastest Route Solver (DFS)	63
5.3.5	Motion Planning (Action Queuer)	63
5.3.5.1	Motion Primitives	63
5.3.5.2	Trajectory Optimization Pipeline	64
5.3.5.3	Dynamic Action Compilation	64
5.3.6	Trajectory Generation and Curve Profiling	64
5.3.6.1	Mathematical Derivation of Curve Dynamics	65
5.3.6.2	Trapezoidal Angular Velocity Profile	65
5.3.6.3	Numerical Approximation for Real-time Calculation	66
5.3.6.4	Solution	66
5.3.7	Control Systems	68
5.3.7.1	Speed Controller	68
5.3.7.2	Wall Following and Odometry Correction	69
5.4	Tests and Evaluation	70
5.4.1	Computational Validation: Simulation and Legacy Hardware	70
5.4.2	Electronic Validation: Manufacturing and Signal Integrity	71
5.4.3	Mechanical Validation: Fit and Function	72
5.4.4	Comparative Analysis: Micras v1 vs. Micras v2	73

6	FINAL CONSIDERATIONS	75
6.1	Conclusions	75
6.1.1	Contributions	76
6.1.2	Prospects for Continuity	76
6.1.2.1	Short-Term Integration and Validation	77
6.1.2.2	Electronics and Hardware	77
6.1.2.3	Mechanics and Actuation	77
6.1.2.4	Computation and Software	78
	BIBLIOGRAPHY	79

1 Introduction

This chapter establishes the fundamental pillars guiding the development of this work. The first section addresses the project's motivation and context, starting from a functional prototype to identify the need for a systematic evolution. The objectives section outlines the specific goals to be achieved, defining the technical and functional parameters of the redesigned robot across mechanical, electrical, and computational domains. Finally, the justification section presents arguments substantiating the relevance and complexity of this work, demonstrating its merit as an advanced engineering project and a significant contribution to the robotics community.

1.1 Motivation

Competitive robotics represents fertile ground for innovation and the development of practical engineering skills. Among various modalities, the Micro Mouse category stands out for its technical challenges, requiring a high degree of integration between mechanical, electrical, and computational systems to create autonomous robots capable of solving complex mazes in the shortest possible time. Although consolidated internationally, the category is still incipient in Brazil, presenting an opportunity for the development of new technologies and the training of skilled professionals.

In this scenario, the "Micras" project ([BARBOSA; PEREIRA, 2024](#)) was developed: a Micromouse robot that served as a foundational prototype. This first effort culminated in an expressive result: third place at RoboChallenge 2024, the first official competition of the category in Brazil. Success in a competitive environment validated the feasibility of the initial concept and the integrated operation of its subsystems.

However, the most valuable outcome of the first version was not its ranking, but the empirical data and critical analysis provided by the competition experience. Real-world operation exposed intrinsic limitations of the project that were not evident in controlled tests. While structural inconsistencies, such as excessive gear backlash, affected stability, critical bottlenecks were identified in the computational architecture and electronics. The adopted control strategies and odometry techniques proved inefficient in handling robot dynamics at high speeds, resulting in instability and trajectory error accumulation. Additionally, electronic design flaws introduced noise into sensor readings, hindering firmware response.

Therefore, the motivation for this new work stems directly from this systemic analysis. The first version of Micras fulfilled its role as a validation and diagnostic platform,

generating a well-defined set of engineering problems. This project represents the second iteration in the development cycle, a direct and systematic response to the observed deficiencies, aiming to transform a successful prototype into a high-performance platform capable of supporting advanced control algorithms and intelligent navigation.

1.2 Objective

The primary objective of this work is to execute the systematic redesign and integral refactoring of the Micras robotic platform, aiming to transform a functional prototype into a reference system characterized by high performance, reliability, and robustness. The project seeks to transcend the limitations of an initial model to create a robot capable of competing at the highest level of the Micromouse category.

To achieve this standard, the work establishes specific objectives across three engineering fronts that, when integrated, compose the final solution:

- **Mechanical Scope:** To optimize the robot's physical architecture focusing on miniaturization and structural rigidity. The goal is to eliminate mechanical inaccuracies through high-fidelity manufacturing methods and robust fixation systems, ensuring agile, precise, and repeatable locomotion.
- **Electrical Scope:** To elevate embedded computational capacity and ensure system signal integrity. The objective is to provide a hardware platform capable of processing complex algorithms in real-time, correcting previous design flaws, and ensuring operation immune to noise and electrical instabilities.
- **Computational Scope:** To refactor the software architecture to implement advanced navigation intelligence. The aim is to transition from reactive controls to predictive strategies and implement continuous mapping algorithms, allowing the robot to make optimal trajectory decisions and explore the maze with maximum efficiency.

Beyond technical goals, this work has a strategic objective to contribute to the national academic and robotics community. By documenting and making the project available as an open-source platform, it seeks to lower the entry barrier for new competitors and provide an engineering benchmark for the development of the category in Brazil.

1.3 Justification

The relevance of this work is grounded in the inherent complexity of competitive robotics and the need to foster national technological development. The Micromouse

category serves as an excellent case study for engineering, requiring the synthesis of multidisciplinary knowledge—mechanics, electronics, and computing—into a critical real-time system. However, despite its international consolidation, the category still lacks widespread dissemination and accessible technical references in Brazil.

Although robot projects are available in the community, there is a significant gap in the quality of documentation and software architecture presented. Many winning projects operate as "black boxes" or use rigid, poorly modular code structures, which hinders study and evolution by new students. There is, therefore, a need for a project that not only functions but is also transparent, well-architected, and extensible.

In this context, the redesign of the Micras platform is justified by the technical necessity to overcome the physical limitations of the previous prototype to enable the practical implementation of refined control strategies. The first version, while functional, restricted the use of high-performance algorithms due to mechanical instabilities and electrical noise. By developing a robust platform with precise fixtures and reliable electronics, this work allows for the validation and effective use of techniques such as feed-forward control and continuous navigation, which would be unfeasible or unstable on severely limited hardware.

Thus, the contribution of this work transcends the construction of the robot itself. By providing an open-source project with an emphasis on software architecture quality and the documentation of engineering decisions, the aim is to lower the entry barrier for new competitors and provide a solid benchmark for the Brazilian academic community.

1.4 Organization of the Work

This monograph is structured into six main chapters, organized to present the logical evolution of the project, from theoretical conception to technical implementation and results analysis.

Chapter 2 (Conceptual Aspects) establishes the theoretical foundation necessary for understanding the project. It addresses the rules and specificities of the Micromouse category, reviews fundamental concepts of control systems and odometry, and presents the state of the art in maze navigation algorithms.

Chapter 3 (Work Method) describes the engineering methodology applied to the redesign. It details the failure analysis approach of the previous prototype, the development cycle adopted for the modular refactoring of subsystems, and the incremental validation strategies (simulation and bench tests).

Chapter 4 (Requirements Specification) formally defines the project scope. Functional and non-functional requirements are presented, segregated into mechanical, electrical,

and computational fronts, establishing the technical criteria the platform must meet to be considered high-performance.

Chapter 5 (Work Development) constitutes the technical core of the monograph. It details the transformation of requirements into a solution, describing: the technologies and tools used; the mechanical design and manufacturing; the electronics design (schematics and PCB layout); and the implementation of the firmware architecture, including control and intelligence algorithms.

Finally, Chapter 6 (Final Considerations) presents the project assessment. It discusses conclusions regarding the achieved objectives, details the original contributions to the open-source community, and points out perspectives for future work in the field.

2 Conceptual Aspects

2.1 Micro Mouse Category

Micromouse is an international robotics competition where autonomous robots must navigate and solve a maze in the shortest possible time. Originally established by the IEEE in the late 1970s, the category has evolved into a benchmark for embedded systems, control theory, and path-planning algorithms.

Unlike remote-controlled categories, the fundamental challenge of Micromouse lies in the complete autonomy of the system. The robot must perceive its environment, map an unknown topology, calculate the optimal trajectory, and execute high-speed movements using limited onboard processing power and energy resources.

2.1.1 The Robot: Physical and Functional Constraints

The rules impose strict physical constraints that directly influence the mechatronic design. The robot's projection must be contained within a square of 25×25 cm (NTF, 2025). However, the physical reality of the maze imposes much tighter limits for high-performance competitors.

Since the maze cells are 18×18 cm (NTF, 2025), a robot close to the maximum allowed size would be unable to perform rotational maneuvers or execute diagonal trajectories within the corridors. Therefore, competitive designs prioritize severe miniaturization. This drives the need for high-density PCB design and compact mechanical transmission, justifying the dimension reduction targets (50×90 mm) proposed in this work.

Functionally, the robot must be strictly autonomous. No communication with external computers is allowed during the run. This requires the embedded firmware to handle all tasks, from low-level motor control (PWM generation) to high-level search algorithms (e.g., Flood Fill), in real-time, highlighting the importance of the MCU's computational throughput.

2.1.2 The Environment: Maze Specifications

The maze is composed of a 16×16 grid of unit squares. The walls are 5 cm high and 1.2 cm thick, leaving a path width of 16.8 cm (NTF, 2025).

The optical properties of the maze are standardized: walls are white, tops are red, and the floor is matte black. These characteristics dictate the design of the wall-sensing

subsystem. The use of infrared (IR) emitter-receiver pairs takes advantage of the high reflectivity of the white walls versus the low reflectivity of the black floor (or empty space) to estimate lateral distance and alignment.

The start zone is located at one of the corners, and the goal zone is a 2×2 open area located at the center of the maze. The robot must identify these zones solely through its onboard sensors and mapping logic.

2.1.3 Competition Dynamics: Exploration vs. Speed Run

A standard competition match allows each robot a total time of 7 to 10 minutes (depending on the specific tournament rules) to perform up to 5 runs (NTF, 2025). This structure divides the robot's operation into two distinct phases:

Exploration Phase: The robot navigates cautiously to map the maze walls and locate the goal. The priority here is safety and data integrity rather than raw speed. Algorithms like Flood Fill or DFS (Depth-First Search) are commonly used to ensure the goal is found. **Speed Run Phase:** Once the map is acquired (or sufficient paths are known), the robot computes the fastest trajectory from Start to Goal. In this phase, the robot accelerates to its physical limits, cutting corners and potentially using diagonal paths (diagonal runs) to minimize travel distance and time.

The final score is based on the fastest run time. This dynamic imposes a critical requirement on the software architecture: it must support persistent memory (to retain the map between runs) and distinct state behaviors (switching from conservative exploration control parameters to aggressive speed-run parameters).

2.2 Fundamentals of Mobile Robotics

Autonomous robot navigation requires a mathematical model that relates the control commands sent to the actuators with the resulting physical behavior in space. For the Micromouse category, where trajectory precision is critical, understanding kinematics and position estimation techniques is fundamental.

2.2.1 Differential Kinematics

The mechanical architecture adopted in this project is the Differential Drive. In this configuration, the robot features two drive wheels mounted on a common axis, controlled independently.

This topology is widely used in the category due to its mechanical simplicity and the ability to perform rotations around its own geometric center (zero turning radius), an

essential feature for agile maneuvers within the square cells of the maze.

The kinematic model relates the angular velocities of the left (ω_L) and right (ω_R) wheels to the robot's linear (v) and angular (ω) velocities in the inertial frame. Considering r as the wheel radius and L as the distance between the wheel contact points (track width), the forward kinematics is given by (SIEGWART; NOURBAKHSH; SCARAMUZZA, 2011):

$$\begin{aligned} v &= \frac{r}{2}(\omega_R + \omega_L) \\ \omega &= \frac{r}{L}(\omega_R - \omega_L) \end{aligned}$$

For control purposes, the inverse kinematic model is required, which converts the robot's desired velocities (v_{ref}, ω_{ref}) into the rotation speeds that must be applied to each motor:

$$\begin{aligned} \omega_R &= \frac{v_{ref} + \frac{L}{2}\omega_{ref}}{r} \\ \omega_L &= \frac{v_{ref} - \frac{L}{2}\omega_{ref}}{r} \end{aligned}$$

These equations assume non-slip conditions both laterally and longitudinally, an idealization that, in practice, is challenged by the high accelerations required during a "Speed Run."

2.2.2 Odometry and Pose Estimation

Odometry is the process of estimating the change in the robot's position over time by integrating movement data from actuators. In mobile robotics, this technique is also known as Dead Reckoning (BORENSTEIN; EVERETT; FENG, 1996).

The robot's pose in a 2D plane is defined by the state vector $q = [x, y, \theta]^T$, where (x, y) represent Cartesian coordinates and θ represents the heading (orientation). In a discrete digital system, the position update at each time interval Δt is calculated by integrating the linear (v_k) and angular (ω_k) velocities measured by the encoders (SIEGWART; NOURBAKHSH; SCARAMUZZA, 2011):

$$\begin{aligned} \theta_{k+1} &= \theta_k + \omega_k \Delta t \\ x_{k+1} &= x_k + v_k \Delta t \cos\left(\theta_k + \frac{\omega_k \Delta t}{2}\right) \\ y_{k+1} &= y_k + v_k \Delta t \sin\left(\theta_k + \frac{\omega_k \Delta t}{2}\right) \end{aligned}$$

The term $\frac{\omega_k \Delta t}{2}$ is a second-order approximation to account for the variation in orientation during the sampling interval itself, offering greater precision in curves compared to simple Euler integration.

2.2.2.1 The Challenge of Accumulative Error (Drift)

Although fundamental, odometry based purely on encoders is prone to unbounded accumulative errors (drift). These errors are classified into (BORENSTEIN; EVERETT; FENG, 1996):

- Systematic Errors: Imperceptible differences in wheel diameters ($rL \neq rR$), uncertainty in the track width measurement (L), and axle misalignment.
- Non-Systematic Errors: Wheel slippage (skidding), floor irregularities, and mechanical backlash.

Since the current position depends on the sum of all previous increments, a small angular error $\delta\theta$ results in a lateral position error that grows proportionally with the distance traveled. This phenomenon justifies the need, addressed in Chapter 5, to implement Local Odometry aided by external sensors (infrared) for constant pose recalibration within the maze cells.

2.3 Control Theory and System Dynamics

High-speed autonomous navigation requires a control system capable of maintaining stability while executing aggressive maneuvers. This section establishes the theoretical framework for the control architectures and physical constraints applied in this work.

2.3.1 Feedback Control Systems (PID)

The classic approach to regulating a robot's motion is the Feedback Loop. In this architecture, the system continuously measures the output variable (e.g., wheel velocity or wall distance), compares it to a desired setpoint, and calculates an error signal $e(t) = r(t) - y(t)$ (OGATA, 2010).

The Proportional-Integral-Derivative (PID) controller is the industry standard for such applications. Its control law is defined as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Where $u(t)$ is the control output, and K_p, K_i, K_d are the tunable gains.

- **Limitations in High-Performance Robotics:** While robust, a pure PID controller is fundamentally reactive. It requires an error to manifest before generating a corrective action. In a Micromouse context, where a robot may travel at 3 m/s, the latency inherent in waiting for an error can lead to significant tracking deviations or collisions before the controller can compensate.

2.3.2 Feed-Forward Control Architecture

To overcome the latency of reactive control, a Feed-Forward term is introduced. Unlike feedback, which reacts to the past/present error, feed-forward predicts the necessary control output based on a mathematical model of the system's physics.

For a DC motor, the voltage V required to achieve a certain state depends on the Back-EMF (proportional to speed) and the Torque (proportional to acceleration). A theoretical feed-forward controller calculates the output u_{ff} directly from the reference setpoint:

$$u_{ff}(t) = K_v \cdot v_{ref}(t) + K_a \cdot a_{ref}(t) + K_{static}$$

Where K_v accounts for velocity-dependent forces (friction, back-EMF) and K_a accounts for inertia. The final control signal sent to the motors is the sum of the predictive term and the corrective term:

$$u_{total}(t) = u_{ff}(t) + u_{PID}(e(t))$$

This hybrid architecture allows the robot to execute aggressive acceleration profiles with minimal error, as the PID term only needs to compensate for disturbances and modeling inaccuracies, rather than driving the entire motion.

2.3.3 Motion Profiling

Robots cannot change velocity instantaneously due to mechanical inertia and motor torque limits. Sending a step command (e.g., from 0 to 1 m/s) results in wheel slip and loss of odometry.

Motion Profiling is the generation of a kinematic trajectory that respects the physical constraints of the system. The most common profile used in robotics is the Trapezoidal Velocity Profile:

- **Acceleration Phase:** Velocity increases linearly with constant acceleration ($+a_{max}$).
- **Cruise Phase:** Velocity remains constant at the target speed (v_{max}).

- **Deceleration Phase:** Velocity decreases linearly ($-a_{max}$) until the target position is reached.

This trajectory generator provides the instantaneous references (v_{ref}, a_{ref}) required by the Feed-Forward controller described in section 2.3.2.

2.3.4 Dynamics of Cornering

A critical limiting factor in maze solving is the maximum speed at which the robot can negotiate a turn. This limit is dictated by the laws of physics, specifically the relationship between centripetal force and tire friction.

When a robot of mass m executes a turn of radius R at a tangential velocity v , it is subject to a centripetal force F_c directed towards the center of curvature:

$$F_c = \frac{m \cdot v^2}{R}$$

This force must be supplied by the lateral friction between the tires and the maze floor. The maximum available friction force $F_{f_{max}}$ is defined by the Coulomb friction model:

$$F_{f_{max}} = \mu N$$

Where μ is the coefficient of static friction between the tire compound and the floor surface, and N is the normal force. In a standard robot relying solely on gravity, $N = mg$. Thus, the maximum velocity is limited by:

$$\frac{m \cdot v_{max}^2}{R} = \mu mg \Rightarrow v_{max} = \sqrt{\mu g R}$$

2.3.5 Aerodynamic Downforce Enhancement

To overcome this gravitational limit, high-performance Micromouse designs often incorporate active suction systems to generate aerodynamic downforce (F_{down}). This system creates a low-pressure zone underneath the chassis, adding an external vertical force to the normal load without significantly increasing the robot's inertial mass.

The effective normal force becomes:

$$N_{total} = mg + F_{down}$$

Substituting this into the friction limit equation:

$$\frac{m \cdot v_{max}^2}{R} \leq \mu(mg + F_{down})$$

Solving for velocity reveals the performance gain:

$$v_{max} = \sqrt{\mu R \left(g + \frac{F_{down}}{m} \right)}$$

This derivation demonstrates mathematically that by artificially increasing the normal force through suction, the robot can achieve cornering speeds significantly higher than those dictated by gravity alone, justifying the inclusion of the suction unit described in the hardware implementation.

2.4 Environment Representation and Search Algorithms

The maze solving problem can be fundamentally modeled as a graph traversal problem. To understand the strategies applied in autonomous navigation, it is necessary to review the underlying concepts of environment representation and the algorithmic classes used for search and optimization.

2.4.1 Graph-Based Representation

In computer science, a maze is typically abstracted as a Graph $G = (V, E)$.

- **Vertices (V):** Represent the discrete units of space (cells). In a standard Micromouse maze, this forms a Grid Graph where each node represents a coordinate (x, y) .
- **Edges (E):** Represent the connectivity between adjacent cells. The existence of a wall implies the removal of an edge between two vertices.

Graphs can be **Weighted** (where edges carry different costs, e.g., distance or time) or **Unweighted** (where all edges have a uniform cost of 1).

2.4.2 Breadth-First Search (BFS) and Flood Fill

Breadth-First Search (BFS) is a fundamental algorithm for traversing graph data structures. It starts at a root node and explores all neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.

- **Optimality:** BFS is mathematically guaranteed to find the shortest path (minimum number of edges) in an unweighted graph.

Flood Fill is a variation of BFS commonly applied in grid-based environments to generate a distance map or gradient field relative to a target node. Ideally, it functions as a BFS expanding from the goal cells backwards to the entire accessible grid. The algorithm serves two primary theoretical purposes in autonomous navigation:

- **Gradient Descent Navigation:** It provides a greedy heuristic for decision-making. By assigning a distance value to every cell, the algorithm creates a potential field where the agent can determine the optimal move from any location simply by transitioning to the neighbor with the lowest value.
- **Dynamic Topology Update:** In scenarios where the graph structure is initially unknown, the discovery of obstacles (walls) implies the removal of edges. The Flood Fill algorithm supports dynamic recalculation of the distance field to reflect these topological changes, ensuring the agent adapts to new constraints and avoids local minima (dead ends).

2.4.2.1 Depth-First Search (DFS)

Depth-First Search (DFS) is an algorithm that explores as far as possible along each branch before backtracking. Unlike BFS, which expands radially, DFS dives deep into the graph structure using a stack (LIFO) or recursion.

- **Characteristics:** DFS is generally more memory efficient than BFS for deep graphs but does not guarantee finding the shortest path in its naive implementation. It typically finds a solution (if one exists) quickly, but not necessarily the optimal one.
- **Application:** DFS is often used in scenarios where the complete path needs to be analyzed recursively, or when enumerating all possible paths is required for optimization tasks.

2.4.2.2 Optimization Techniques: Branch and Bound

To solve optimization problems where the search space is large (such as finding the fastest trajectory in a complex maze with weighted turns), exhaustive search methods become computationally prohibitive. Branch and Bound is a general algorithm design paradigm used to solve combinatorial optimization problems.

The method enumerates candidate solutions (branches) but discards (prunes) branches that cannot possibly yield a better solution than the best one found so far.

- **Pruning:** If the cost of the current partial path $g(n)$ exceeds the cost of the best complete path found (C_{best}), the algorithm stops exploring that branch immediately ($g(n) \geq C_{best}$).

- Heuristics: Efficiency can be further improved by adding a heuristic function $h(n)$ that estimates the minimum cost to reach the goal. A branch is pruned if $g(n) + h(n) \geq C_{best}$. For grid maps, the Manhattan Distance ($|x1 - x2| + |y1 - y2|$) is a common admissible heuristic, as it never overestimates the cost in an orthogonal grid (RUSSELL; NORVIG, 2010).

3 Methodology

3.1 Development Planning

Based on the objectives established in the previous chapter, this project aims to refactor and optimize a Micromouse robotic platform. To achieve this goal, the project follows a methodology structured into sequential and interdependent steps, ranging from the diagnosis of the previous version to the validation of the new system.

3.1.1 Critical Performance Analysis

The first step consists of a rigorous analysis of the previous robot iteration (Micras v1). This diagnosis relies on empirical observation of behavior during the RoboChallenge 2024 and direct debugging sessions. The goal is to identify specific failure points in mechanics (such as backlash), electronics (thermal instability in drivers), and logical decision-making that prevented the robot from reaching its theoretical top speed.

3.1.2 Requirements Definition and Specification

Following the diagnosis, the specific technical requirements for the new version (v2) will be established. This step defines the physical constraints (maximum dimensions), electrical parameters (voltage levels, current capacity), and computational metrics (control loop frequency) required to overcome the identified limitations.

3.1.3 High-Level Software Refactoring and Simulation

Given the complexity of the navigation algorithms, the software development is decoupled from the physical hardware production. This is made possible by the project's layered software architecture, which was already present in v1. The system utilizes a Hardware Proxy ([DOUGLASS, 2011](#)) pattern to abstract low-level peripherals. Therefore, the High-Level logic (responsible for mapping and path planning) does not interact directly with the hardware, making the underlying MCU migration transparent to the navigation stack.

In this stage, the High-Level algorithms will be updated and validated using physical simulators, specifically MuJoCo ([TODOROV; EREZ; TASSA, 2012](#)) and a custom fork of the Bots2d simulator ([ARTFULBYTES, 2025](#)). This allows the logic to be stress-tested in a virtual environment before deployment. Additionally, due to the abstraction layers, the

new High-Level logic can be deployed and tested on the v1 hardware to validate logical behavior in the real world, even before the v2 hardware is available.

3.1.4 Mechatronic Co-Design

The hardware design follows a co-design methodology, where mechanical and electronic developments occur simultaneously. Since the Printed Circuit Board (PCB) serves as the robot's structural chassis, the two domains are intrinsically linked.

- **Mechanical Design:** The mechanical CAD defines the dimensional constraints, motor positions, and sensor layouts.
- **Electronic Design:** The schematic and PCB layout are developed based on the mechanical outline. Constraints found during routing (e.g., component height, connector placement) are fed back into the mechanical design, creating an optimization loop until the final form factor is frozen.

3.1.5 Manufacturing and Hybrid Assembly

The fabrication strategy prioritizes a balance between precision and feasibility. For the electronics, the PCB will be manufactured and partially assembled (SMD components) by a specialized service (JLCPCB) to ensure reliability. Remaining components, such as through-hole connectors and specific sensors, will be manually soldered in-house. For the mechanics, complex structural parts like motor mounts and gears will be manufactured using SLA (Resin) 3D printing. This method is chosen to achieve the high resolution and tight tolerances required for interference fits, which are essential to eliminate mechanical play.

3.1.6 System Integration and Validation Plan

The final stage involves the integration of all subsystems. The validation plan is structured in three levels:

- **Bench Testing:** Individual electrical verification of subsystems (regulators, sensors, and motor drivers) to ensure they meet the electrical specifications.
- **Mechanical Alignment:** Assembly of the drive train and verification of wheel alignment and gear mesh quality.
- **Dynamic Validation:** The complete system will be tested in a real maze environment to tune the control loops and verify the efficacy of the navigation algorithms compared to the simulation results.

4 Requirements Specifications

This chapter defines the technical specifications and constraints that guide the redesign of the Micras platform (v2). The requirements are derived from the critical analysis of the previous prototype (v1) and aim to address specific bottlenecks in mechanics, electronics, and computation.

4.1 Functional Requirements

Functional requirements describe the specific behaviors and functions the system must perform, abstracting the implementation details.

- **Autonomous Maze Solving:** The robot must explore a 16×16 grid maze, map the wall topology, and identify the shortest path to the goal zone without external intervention.
- **Optimized Exploration Strategy:** The system must implement a search strategy that prioritizes unvisited areas to maximize map coverage efficiency, minimizing redundant movements during the exploration phase.
- **Kinematic Route Evaluation:** The system must evaluate candidate paths based on estimated travel time, considering the robot's physical constraints (velocity, acceleration, and turn speed) rather than solely geometric distance.
- **Continuous Trajectory Execution:** The robot must be capable of executing complex paths—including diagonal runs and reduced-radius turns—with continuous velocity profiles, avoiding complete stops between distinct movement segments.
- **Interface and Debugging:** The robot must provide a wired interface (USB-C) for firmware programming and low-level debugging. Additionally, it must support a wireless link (BLE) dedicated to real-time telemetry, state monitoring, and remote configuration.
- **Power Management:** The robot must include a physical mechanism to safely isolate the battery from the circuit.
- **Localized Error Correction:** The navigation system must utilize maze landmarks to reset accumulated position errors, supporting a localized odometry paradigm.

4.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, physical constraints, and performance metrics.

4.2.1 Mechanical Constraints

- **Locomotion Architecture:** The platform must utilize a 2-wheel differential drive configuration to minimize static indeterminacy and reduce tolerance accumulation.
- **Dimensional Reduction:** The robot's footprint must be minimized (target approx. 50×90 mm) to provide sufficient clearance for diagonal maneuvers.
- **Transmission Ratio:** The gearbox must provide a high reduction ratio (3.8:1) to increase torque availability for rapid acceleration.
- **Assembly Rigidity:** Mechanical components must be fixed using interference fits and screws, avoiding glued assemblies to ensure precision and maintainability.

4.2.2 Electrical Constraints

- **Propulsion Voltage:** The motor drive system must operate at a nominal voltage of 20V to reduce the electrical current required for the target mechanical power.
- **Power Source:** The system must be powered by a 3S LiPo battery to support the high-voltage requirements and improve efficiency.
- **Component Density:** The PCB design must utilize miniaturized component packages (0402/0603) and high-density routing to fit the control system within the reduced mechanical footprint.
- **Driver Robustness:** The suction fan driver must be dimensioned to support high-current operation, preventing thermal shutdowns.

4.2.3 Computational and Control Constraints

- **Processing Power:** The microcontroller must utilize the Cortex-M7 architecture to provide floating-point performance for real-time navigation algorithms.
- **Feed-Forward Control:** The control loop must incorporate predictive model-based terms to improve dynamic response during high-acceleration maneuvers.
- **Action-Scoped Odometry:** The position estimation must operate on a local coordinate system relative to the current action to mitigate long-term drift.

5 Project Development

5.1 Technologies Used

During the product development, specific technologies and tools were selected to ensure precision and manufacturing feasibility. This section presents the most relevant tools utilized in the development process.

5.1.1 Mechanical Design and Manufacturing

The mechanical development of the Micras v2 required tools capable of handling high dimensional precision and tight integration with electronic components.

5.1.1.1 SolidWorks

SolidWorks is a computer-aided design (CAD) and computer-aided engineering (CAE) software program that runs on Microsoft Windows. It uses a parametric feature-based approach to create models and assemblies. In this project, SolidWorks was the primary tool for modeling the robot's structure. Beyond individual part design, its Assembly environment was critical for the mechatronic co-design strategy. It allowed for the importation of the PCB 3D model (exported from the electronic design software) to verify the mechanical fit of components, perform interference detection between the motors and the circuit board, and validate the alignment of the gear train before any physical manufacturing took place.

5.1.1.2 Anycubic Photon Mono 4 (10k)

For the physical manufacturing of the robot's parts, the project utilized the Anycubic Photon Mono 4, a high-resolution 3D printer based on MSLA (Masked Stereolithography) technology. Unlike traditional FDM (Fused Deposition Modeling) printers that use melted filament, MSLA uses an LCD screen to mask a UV light source, curing liquid resin layer by layer. This specific machine features a 10k resolution LCD screen. This extreme resolution was a decisive factor in its selection, as the project requirements included the fabrication of custom micro-gears and motor mounts with interference fits. The printer's capability to produce parts with negligible thermal contraction and high surface fidelity was essential to achieve the backlash-free transmission system required for the robot's locomotion.

5.1.1.3 Lychee Slicer

Lychee Slicer is a software tool used to convert digital 3D models (STL/OBJ) into printing instructions (sliced layers) for resin printers. It allows users to manipulate the model's orientation, generate support structures, and define resin exposure times. In the context of this work, Lychee Slicer was used to optimize the print orientation of mechanical parts to minimize warping forces during the peeling process. It was also used to calibrate the exposure settings specifically for the engineering resin employed, ensuring that the cured parts maintained dimensional accuracy within the tolerances defined in the CAD phase, which is crucial for the assembly of the bearings and shafts.

5.1.2 Electronic Design and Manufacturing

The electronic development of the Micras v2 involved the design of a high-density mixed-signal circuit. To achieve the required miniaturization and reliability, professional-grade tools and services were employed.

5.1.2.1 Altium Designer

Altium Designer is a premier Electronic Design Automation (EDA) software package that unifies schematic capture, PCB layout, and design data management into a single environment. It is widely recognized in the industry for its advanced routing capabilities and 3D visualization features. In this project, Altium Designer was utilized to engineer the complete control board. Its hierarchical schematic features allowed for the modular design of power, sensing, and control subsystems. Crucially, its ECAD-MCAD integration capabilities were leveraged to export STEP files of the populated PCB. These files were imported into SolidWorks to verify mechanical clearance within the robot's chassis, ensuring that tall components (like capacitors) did not interfere with the motors or structural parts, effectively enabling the mechatronic co-design methodology described in Chapter 3.

5.1.2.2 JLCPCB (Manufacturing and Assembly)

JLCPCB is a leading PCB prototype manufacturer and assembly service provider. They utilize automated industrial processes for both board fabrication and surface mount technology (SMT) assembly. For the Micras v2, JLCPCB was selected not just for board fabrication, but for their PCBA (PCB Assembly) service. A hybrid assembly strategy was adopted to optimize costs and leverage available resources. JLCPCB performed the automated soldering of high-density passive components (0402 package) and standard ICs, ensuring reliability where manual soldering would be prone to errors. However, specific high-value components were intentionally excluded from the automated process. Notably,

the STM32 microcontroller was obtained as a free sample directly from STMicroelectronics, and other specific connectors were repurposed from the team's existing inventory. These components were subsequently soldered manually in-house, completing the assembly.

5.1.3 Computation and Firmware

The software stack for the Micras v2 was built upon a modern toolchain, prioritizing code modularity, version control, and rigorous testing capabilities.

5.1.3.1 Visual Studio Code (VS Code)

VS Code served as the primary Integrated Development Environment (IDE) for the project. Unlike proprietary vendor IDEs, VS Code provides a lightweight and highly extensible environment. In this workflow, it was used not only for editing C++ source code but as the central hub for the entire toolchain. Through extensions like Cortex-Debug and CMake Tools, it integrated compilation, flashing, and debugging processes into a single interface, streamlining the iterative development cycle.

5.1.3.2 Git & GitHub

Git was employed as the version control system, hosted on the GitHub platform. Given the open-source nature of the project and the need for frequent refactoring of the high-level architecture, Git was essential for managing change history. It allowed for the isolation of new features (feature branches) before merging them into the main codebase, ensuring a stable deployment version while experimenting with new algorithms.

5.1.3.3 STM32Cube Ecosystem

The project leveraged STMicroelectronics' suite of tools to accelerate hardware configuration and validation:

- **STM32CubeMX:** Used for the graphical configuration of the microcontroller's peripherals (timers, ADCs, SPI). It generated the initialization code and the Hardware Abstraction Layer (HAL), which served as the low-level foundation for the project's custom Hardware Proxy.
- **STM32CubeMonitor:** A non-intrusive monitoring tool used for real-time visualization of variables. It allowed for the plotting of sensor data and internal control states (like PID error or current velocity) while the robot was in motion, without halting the CPU, which is critical for tuning dynamic control loops.
- **STM32CubeProgrammer:** Utilized for low-level memory operations, such as full chip erasure and direct binary flashing via the system bootloader when necessary.

5.1.3.4 Debugging and Programming Toolchain

A robust hardware connection was established to enable code deployment and runtime diagnosis:

- **ST-Link v2:** The primary hardware interface connecting the host computer to the robot via the SWD (Serial Wire Debug) protocol. It is responsible for flashing the compiled firmware binary onto the microcontroller's flash memory and acts as the bridge for debugging data.
- **OpenOCD (Open On-Chip Debugger):** The software middleware that translates GDB (GNU Debugger) commands into JTAG/SWD signals.
- **Cortex-Debug:** A VS Code extension that integrates OpenOCD and GDB. This setup allowed for setting breakpoints, inspecting memory registers, and stepping through instructions directly within the code editor, significantly reducing the time required to diagnose complex logical errors in the firmware.

5.1.3.5 MuJoCo (Physics Engine)

MuJoCo (Multi-Joint dynamics with Contact) is a high-performance physics engine. Before the physical hardware was available, the high-level navigation logic (Flood Fill, DFS, and Path Planning) was validated in a virtual environment simulated by MuJoCo. This tool was chosen for its accurate contact physics and efficient computation, allowing the simulation of robot dynamics (friction, inertia) and sensor ray-casting. This enabled the "High-Level Software Refactoring" phase described in the Methodology, ensuring that the decision-making algorithms were logically sound before deployment on the real microcontroller.

5.2 Project and implementation

5.2.1 Mechanical Implementation

The mechanical redesign was guided by the fundamental need to resolve structural instability and overcome the dimensional limitations of the previous version. The new architecture prioritizes assembly stiffness, severe miniaturization, and precise kinematic behavior. These factors are essential to meet high-performance requirements, such as high-speed cornering and diagonal maneuvering capabilities.

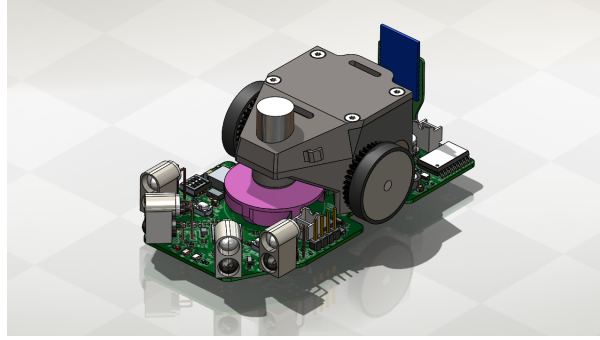


Figure 1 – Isometric view of the complete mechanical design.

5.2.2 Structure and Integration (PCB-as-Chassis)

The robot's architecture adopts the PCB-as-Chassis concept, where the printed circuit board acts as the primary structural element, eliminating the need for an additional carbon fiber or aluminum frame. To ensure a perfectly flat bottom surface, crucial for maximizing the sealing efficiency of the suction system, the motor mount fixation points on the PCB utilize countersunk holes. This allows the use of M2 flat-head screws, which sit flush with the board's surface, preventing drag or turbulence in the underbody airflow.

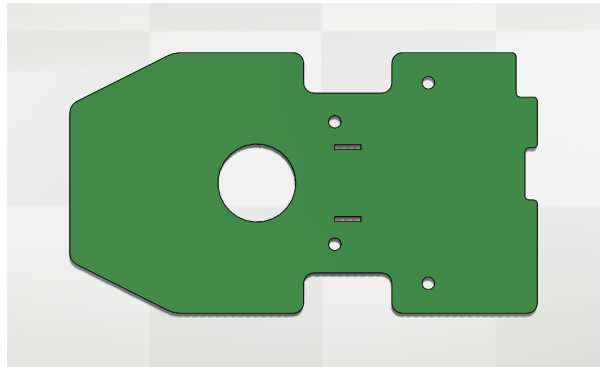


Figure 2 – PCB CAD design.

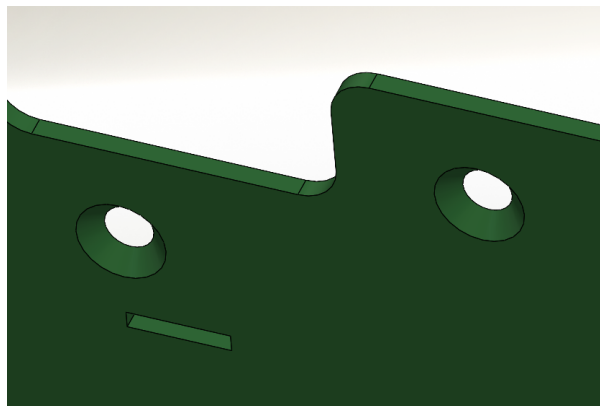


Figure 3 – PCB detail showing countersunk holes for flush mounting.

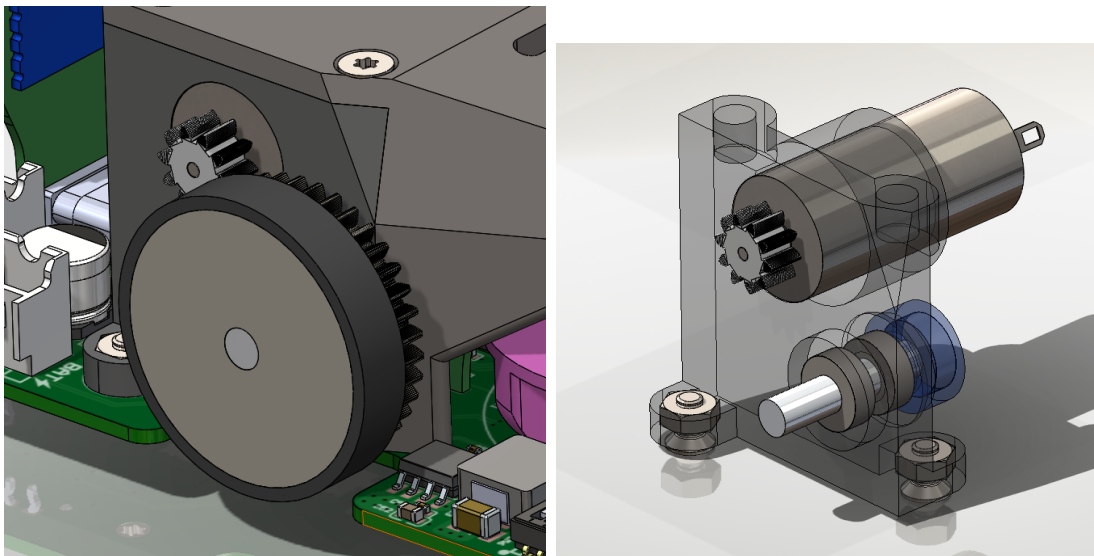
5.2.3 Drive System and Motor Mounts

The motor mount is the critical mechanical component, responsible for the alignment of the entire drivetrain. The design transitioned from a classic 4-wheel configuration to a Two-Wheel topology with Stacked Motors.

This design decision was driven by three factors:

- **Width Reduction:** Since placing two motors side-by-side would exceed the dimensional limits desired for the miniaturized robot, a vertically stacked motor configuration was adopted. This drastically reduced the distance between the mounts, compacting the robot's footprint.
- **Kinematic Simplification:** Reducing the system to two wheels decreases complexity and eliminates static indeterminacy, ensuring the drive wheels maintain constant contact with the ground.
- **Tolerance Mitigation:** With fewer moving parts, the accumulation of printing and assembly errors becomes less significant compared to the v1 iteration.

The high precision of resin printing allowed the motors to be secured via interference fits, eliminating the need for adhesive tape or glue shims, which were common failure points in the previous version. The base of each mount features slots for nuts, enabling a rigid bolted connection to the PCB. The wheel shaft is supported by two precision micro-bearings, ensuring concentricity. The wheel hub is fixed to the outer end of the shaft, while a diametric magnet (used by the AS5047U magnetic encoder) is fixed to the inner end, facing inside the mount.



(a) Zoomed view.

(b) View with transparency.

Figure 4 – Right motor mount.

To complement the miniaturization, the thickness of the silicone tires was reduced by half, maximizing the usable internal volume between the mounts for electronics.

5.2.4 Power Transmission

The transmission system was designed for a gear reduction ratio of 3.8:1, a significant optimization compared to the 2.5:1 ratio used in the previous version. This increase in mechanical advantage provides the necessary torque to ensure the robot can accelerate aggressively and overcome small unevenness or steps between maze tiles without stalling.

The gear specifications are:

- **Pinion:** 10 teeth, 0.5 Module.
- **Spur Gear (Wheel):** 38 teeth, 0.5 Module.

The tooth profile was generated using the STL Gears web tool, which allows for precise parameterization of Pressure Angle and Profile Shift. This enabled the printing of custom resin gears with smooth meshing and low noise.

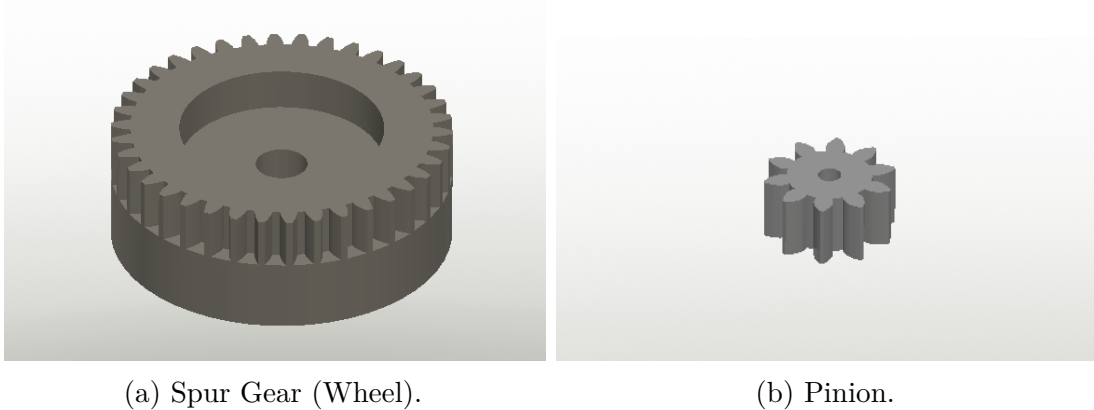


Figure 5 – Transmission set CAD.

5.2.5 Suction System Support (Fan Mount)

The fan mount was designed as a structural component that fastens to the top of the motor mounts via threaded metal inserts. When bolted down, this part bridges the two sides of the robot, forming a truss structure that significantly increases the torsional rigidity of the assembly.

This part was kept separate from the motor mounts (modularity) to facilitate assembly and maintenance. Regarding the suction actuator, the system utilizes a DC motor equipped with a pre-installed pinion gear on its shaft. To couple the custom centrifugal impeller, a matching negative profile was modeled into the fan's hub. The pinion acts as a

spline to transmit torque, while the assembly is permanently bonded using adhesive to ensure structural integrity at high RPMs.

Its function is to generate a low-pressure zone under the robot, creating Downforce. Physically, this increases the Normal Force (N) on the tires without increasing the robot's mass (m), resulting in a higher Friction Force ($F_{friction} = \mu N$), allowing for high-acceleration turns without skidding.

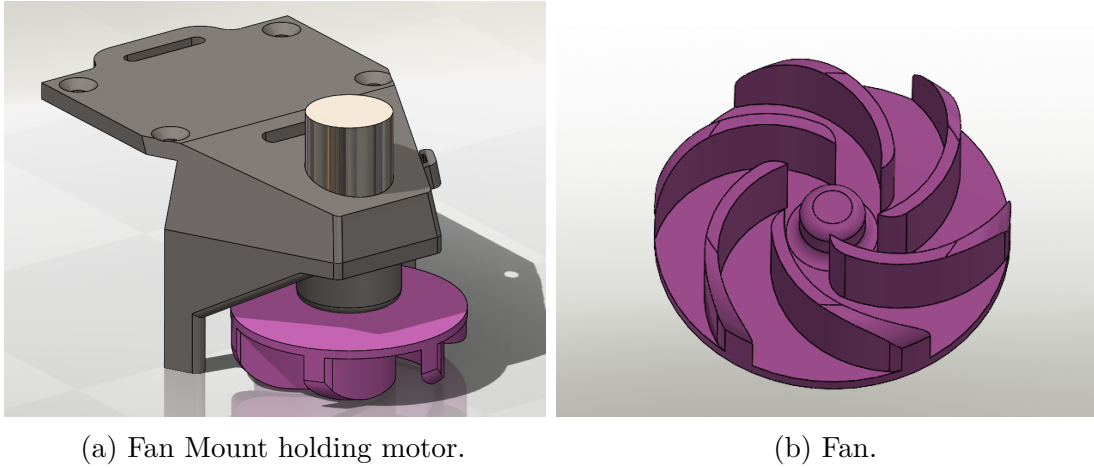


Figure 6 – Fan Mount.

5.2.6 Battery Fixation and Center of Mass

The top of the fan mount features slots designed for velcro straps. This allows the 3S LiPo battery to be firmly secured near the robot's geometric center. This position is strategic for centering the Center of Mass (CoM) and reducing the rotational Moment of Inertia (I), facilitating the PID controller's response during rapid orientation corrections.

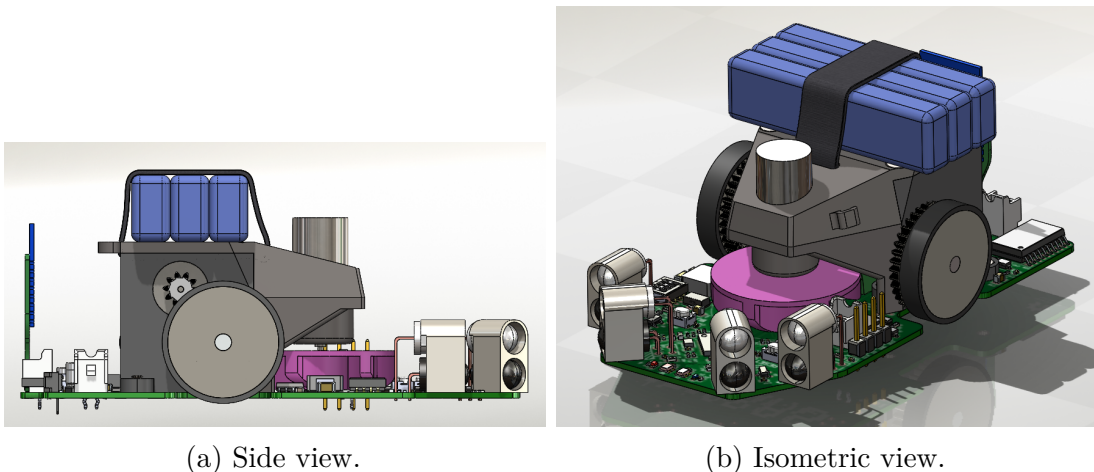


Figure 7 – Battery fixation system aiming for mass centralization.

5.2.7 Manufacturing Process

All mechanical parts were manufactured using the Anycubic Photon Mono 4 (10k) printer. Resin technology was mandatory for this project, as the 10k resolution enabled the fabrication of 0.5 module gear teeth and micrometer-precision interference fits, which would be unfeasible with conventional Fused Deposition Modeling (FDM) printers.

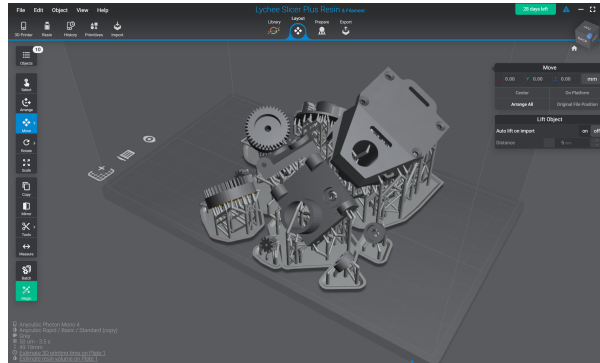
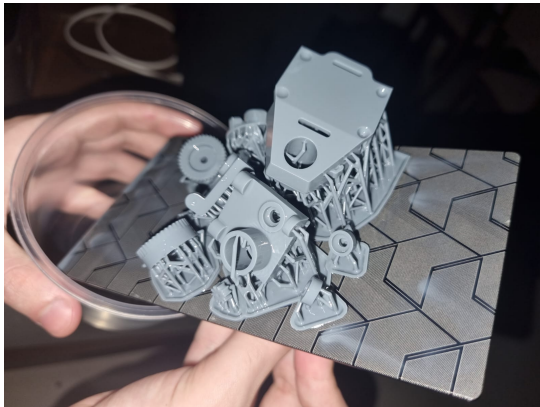


Figure 8 – Part preparation in Lychee Slicer software.



(a) Freshly printed parts.



(b) Parts after curing.

Figure 9 – Parts freshly printed and after the cleaning/curing process.

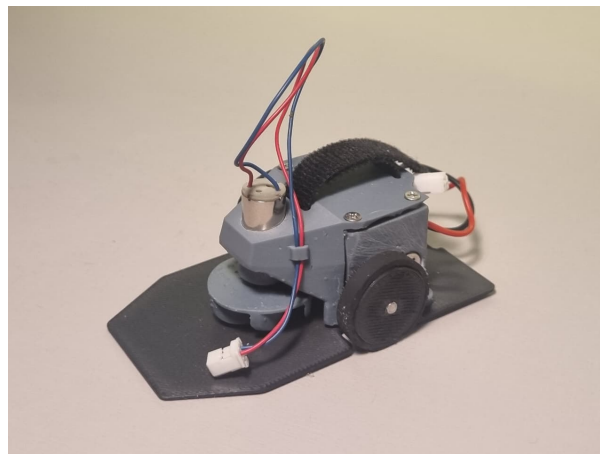


Figure 10 – Fully assembled printed parts.

In alignment with the project’s open-source philosophy and to ensure reproducibility by the academic community, the complete set of mechanical design files (including native SolidWorks assemblies and STL manufacturing files) is hosted publicly at the project’s [GitHub repository](#) (SANTI, 2025).

5.2.8 Electrical Implementation

The robot’s electrical architecture was completely overhauled to meet the rigorous demands of high-speed micromouse competitions. The design philosophy prioritized three key pillars: maximizing computational throughput for real-time control, minimizing PCB footprint, and ensuring power integrity under high dynamic loads.

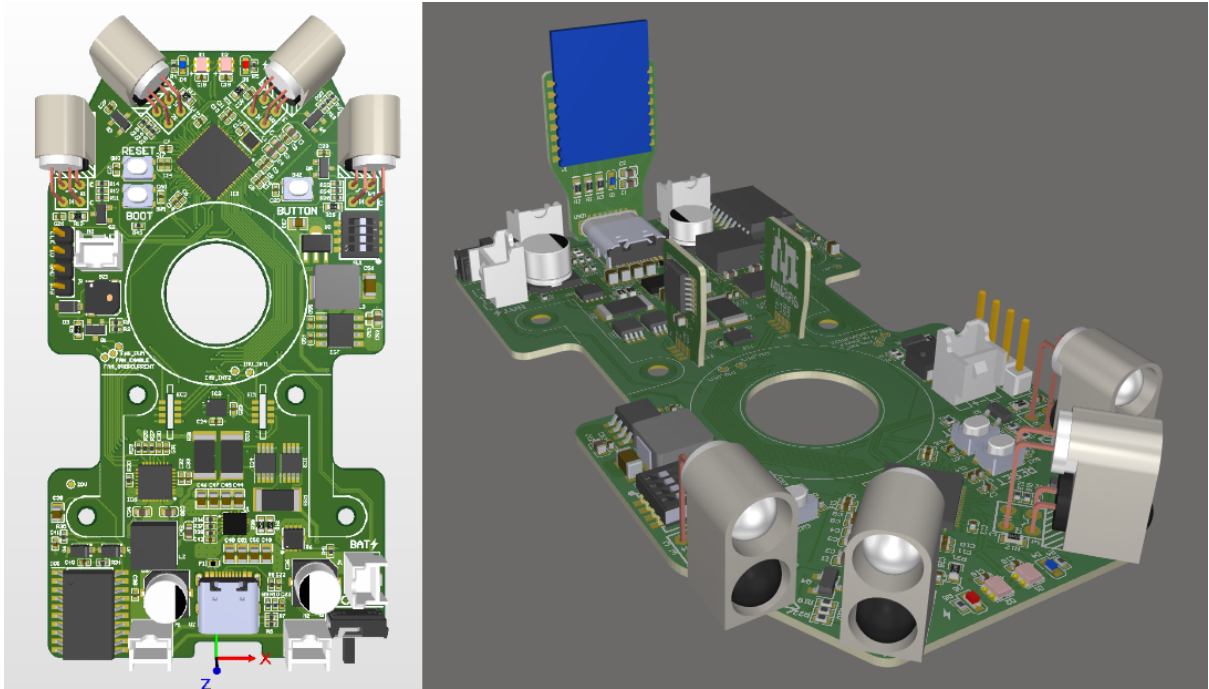


Figure 11 – PCB Altium ECAD project.

5.2.8.1 Microcontroller

The central component of the project is the STM32H725RGV6 (STMICROELECTRONICS, 2025d) microcontroller from STMicroelectronics. This chip features a 32-bit Arm® Cortex®-M7 core with a double-precision FPU, operating at a frequency of up to 550 MHz. The device integrates 1 Mbyte of Flash memory and 564 Kbytes of SRAM, including 192 Kbytes of TCM (Tightly Coupled Memory) for deterministic real-time execution. The package used is the VFQFPN68 (8×8 mm), and the component was configured in Direct SMPS Supply mode to optimize internal power efficiency.

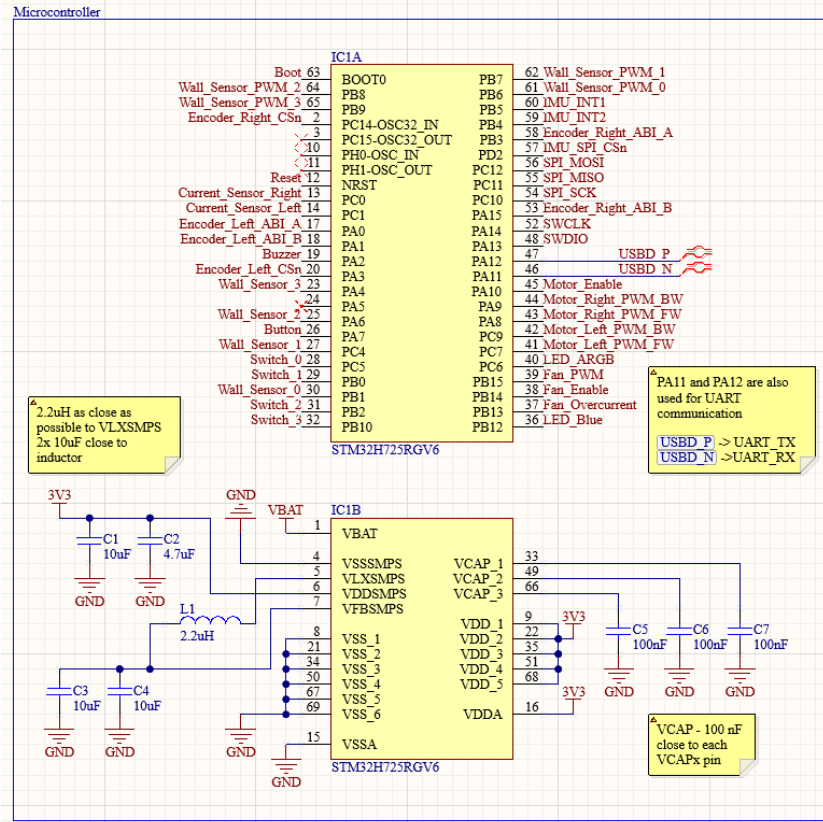


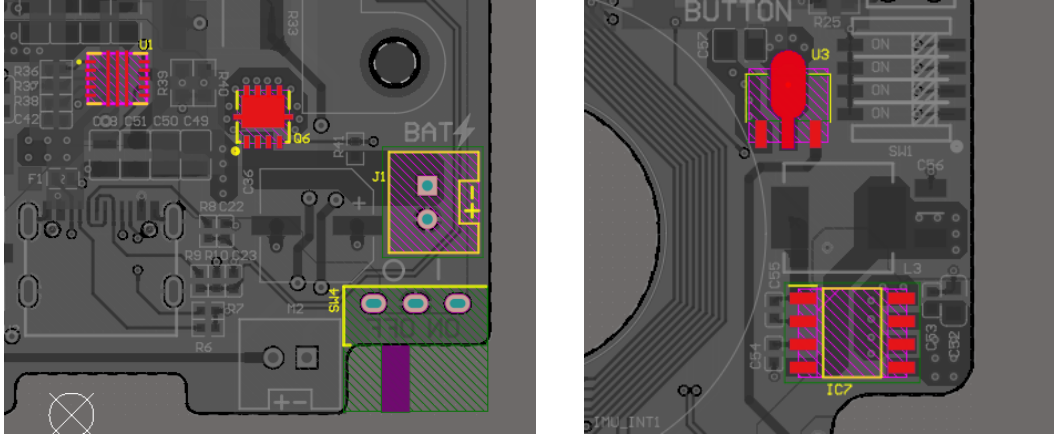
Figure 12 – Microcontroller circuit schematic.

These were the main characteristics considered when selecting the chip. The selection was based on the functional requirements for high-speed navigation and benchmarking against world-class robots. A component offering significantly superior processing capacity compared to the previous version (Cortex-M4) was sought, ensuring that the execution of complex search and motor control algorithms would not be limited by hardware. Additionally, the project execution was facilitated by STMicroelectronics, which provided the microcontrollers as free samples.

5.2.8.2 Power Management Architecture

To ensure system stability under high dynamic loads, the supply voltage was increased from a 2S (7.4V) to a 3-cell LiPo battery (3S, 11.1V). This increase in voltage is strategic: it reduces the current required by the entire circuit to achieve the same power output, thereby improving efficiency and reducing thermal stress on components.

Power switching is implemented via a slide switch coupled to an electronic switching circuit. A HSBB3072 (SINOPOWER, 2025) N-channel MOSFET intercepts the battery's ground (GND) path, acting as a Low-Side Switch. This configuration prevents high currents from flowing directly through the mechanical switch.



(a) Battery switch and boost circuits.

(b) LDO and buck regulator.

Figure 13 – Power management and switching circuit PCB.

Voltage distribution is handled by distinct regulators:

- **Boost Regulator (20V):** The circuit based on the TPS611781RNWR (TI, 2025) IC was redesigned to boost the battery voltage to 20V. This higher voltage rail allows for motor overvolting, significantly increasing the robot's top speed, and enables the use of motors with higher nominal voltages to achieve the same torque and speed targets but with significantly lower current consumption, optimizing battery life and driver thermal performance.
- **Buck Regulator (3.3V):** For digital logic and sensors, the L6981N33DR (STMICROELECTRONICS, 2025b) regulator was retained, stepping down the battery voltage to 3.3V with high efficiency.
- **LDO Regulator (3.3V):** A CJA1117B-3.3 (CO., 2025) LDO was added to regulate the 5V input from the USB-C connector to 3.3V, ensuring the logic remains powered during USB communication even without the main battery.

5.2.8.3 Sensing and Perception

The sensing system received significant mechanical and electronic updates to improve accuracy and reduce noise:

- **Wall Sensors:** The array uses SFH4550 IR emitters (OSRAM, 2025a) and TPS601A phototransistors (TOSHIBA, 2025). Unlike the previous version, this iteration employs a dedicated AO3400A MOSFET (SEMICONDUCTOR, 2025) for each LED to control brightness individually. This topology allows for sequential activation and differential reading (sampling with the LED on and off), facilitating ambient noise

filtering. Furthermore, $1\mu F$ capacitors were placed next to each phototransistor for local high-frequency noise filtering.

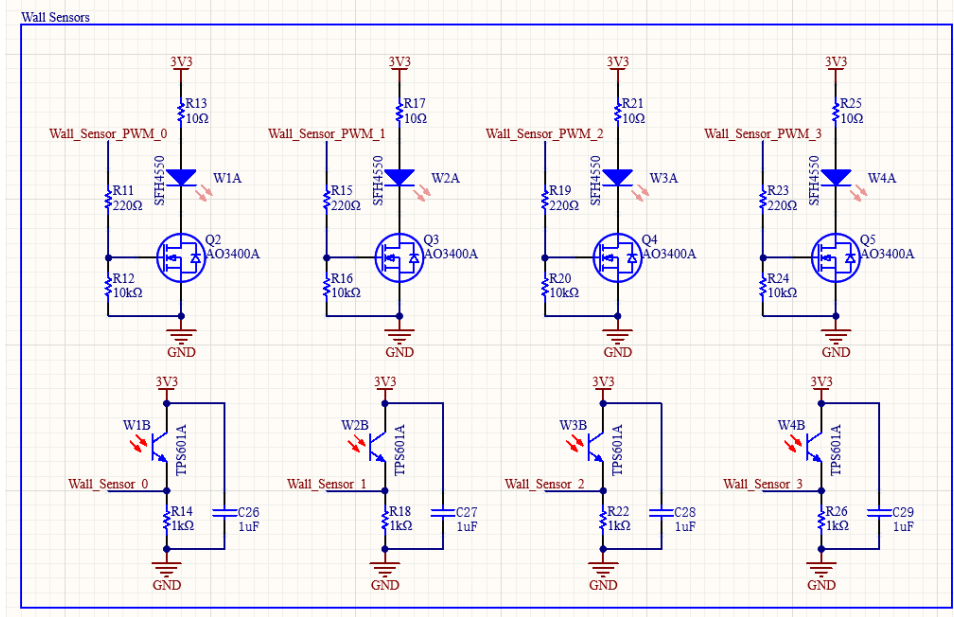


Figure 14 – Wall sensors schematics.

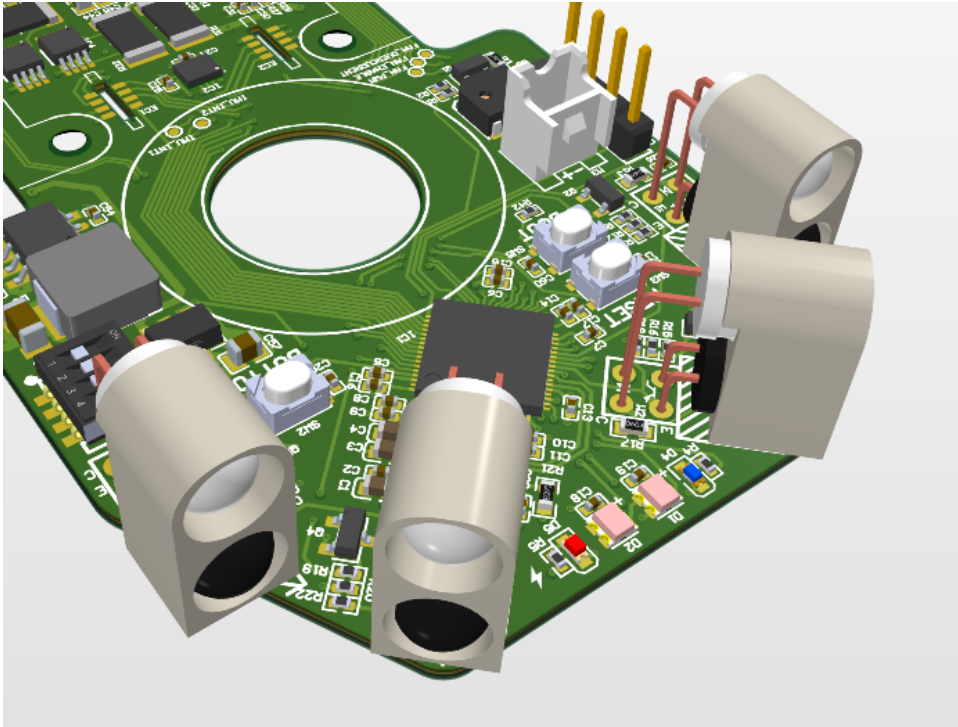


Figure 15 – Wall sensors on PCB.

- **Encoders (Mechanical Optimization):** The AS5047U magnetic IC ([OSRAM, 2025b](#)) was retained, but the mounting method was optimized. The complex soldering of headers was replaced by a direct mating system: the mainboard features a milled

slot where the encoder board slides in and fits perpendicularly. Exposed pads on the slot edges allow for direct soldering, ensuring both electrical connection and robust mechanical fixation.

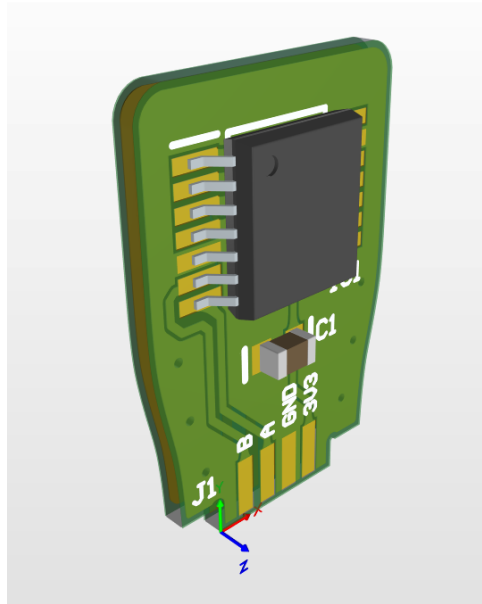
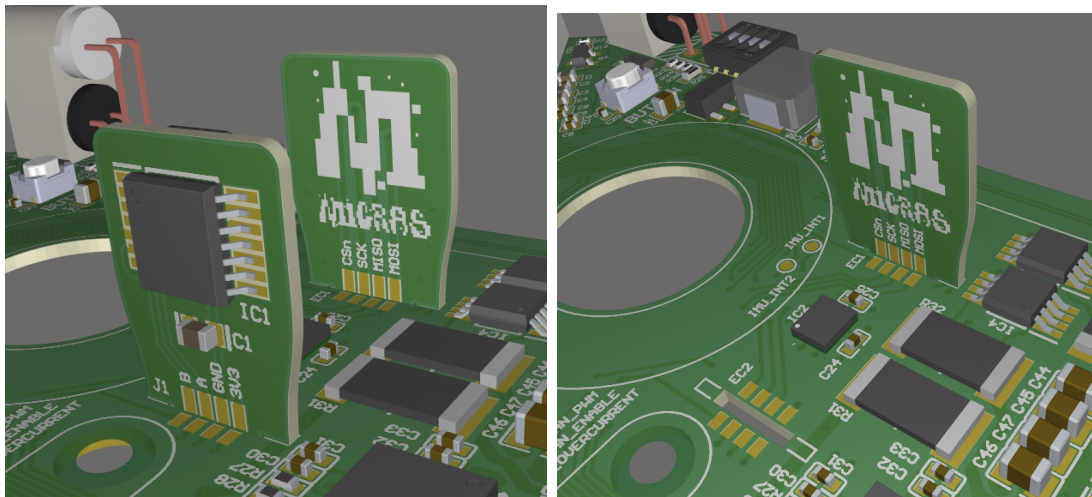


Figure 16 – Encoder board PCB.



(a) Both encoders in place.

(b) Right encoder in place.

Figure 17 – Encoder board mating.

- **Current Monitoring:** The system uses the TSC2020IYST current sense amplifier ([STMICROELECTRONICS, 2025f](#)). The design was improved by changing the voltage reference (V_{ref}) directly to 3.3V and, crucially, implementing Kelvin connections to the shunt resistors, significantly improving the accuracy of motor current readings.

- **IMU:** For inertial orientation, the LSM6DSVTR ([STMICROELECTRONICS, 2025c](#)) was retained, communicating via the SPI bus.

5.2.8.4 Actuation and Drivers

The power stage responsible for movement also underwent revisions focused on electromagnetic stability:

- **Locomotion Motors:** The L6205D ([STMICROELECTRONICS, 2025a](#)) driver was kept, but the circuit was updated to include electrolytic bulk capacitors. Adding these components is vital to handle instantaneous current spikes and to reduce noise injected into the main power rail.

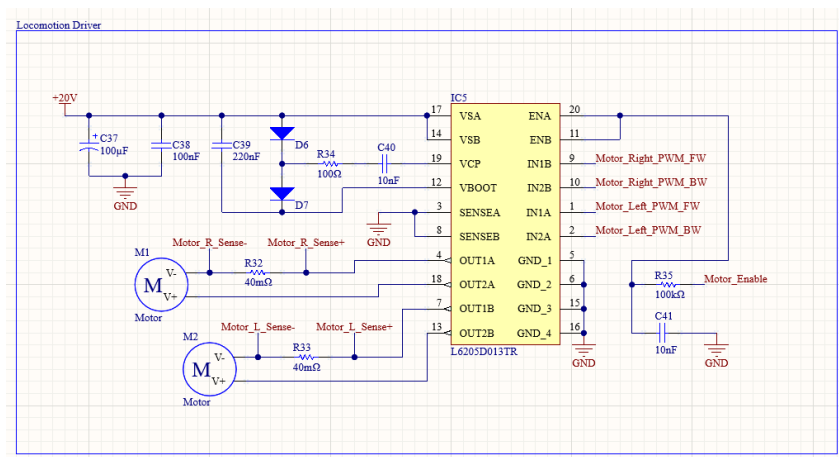


Figure 18 – Locomotion driver schematics.

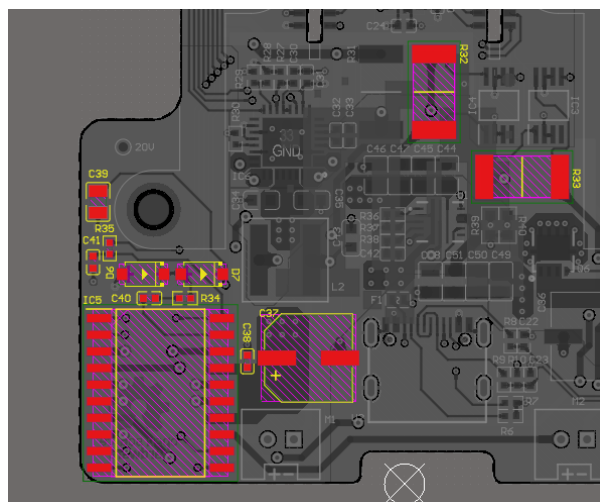


Figure 19 – Locomotion driver PCB.

- **Suction Unit (Fan):** The driver circuit underwent a critical redesign. The previous implementation was undersized for the fan's load, causing the thermal shutdown and

overcurrent protection mechanisms to trigger immediately upon activation, effectively rendering the suction system non-functional. To resolve this, the driver was replaced by the STSPIN958 (STMICROELECTRONICS, 2025e), selected specifically for its high current rating (5A) and overcurrent protection. Additionally, as with the traction motors, a dedicated bulk capacitor was added to the circuit to filter noise and stabilize the voltage supply during high-load operation. This upgrade eliminates the premature protection triggers and, combined with the 3S battery voltage increase, ensures reliable and powerful suction.

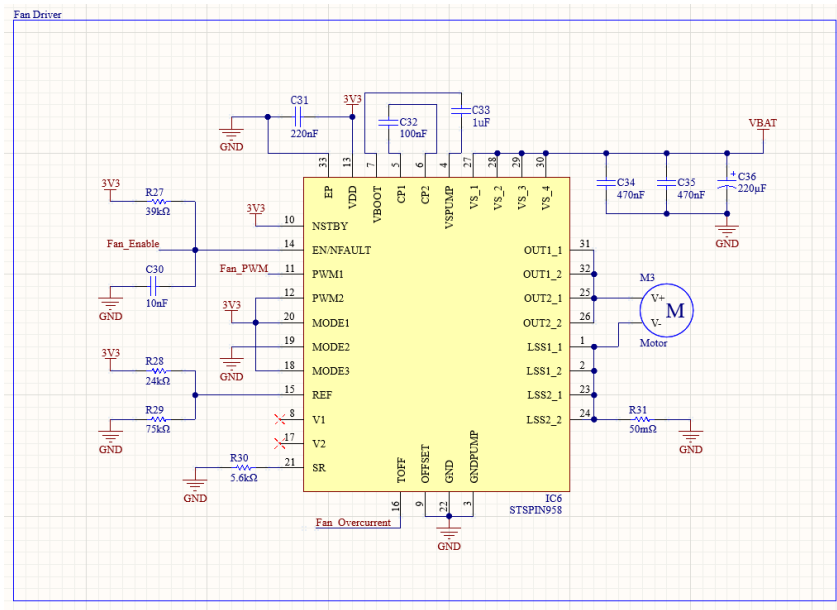


Figure 20 – Fan driver schematics.

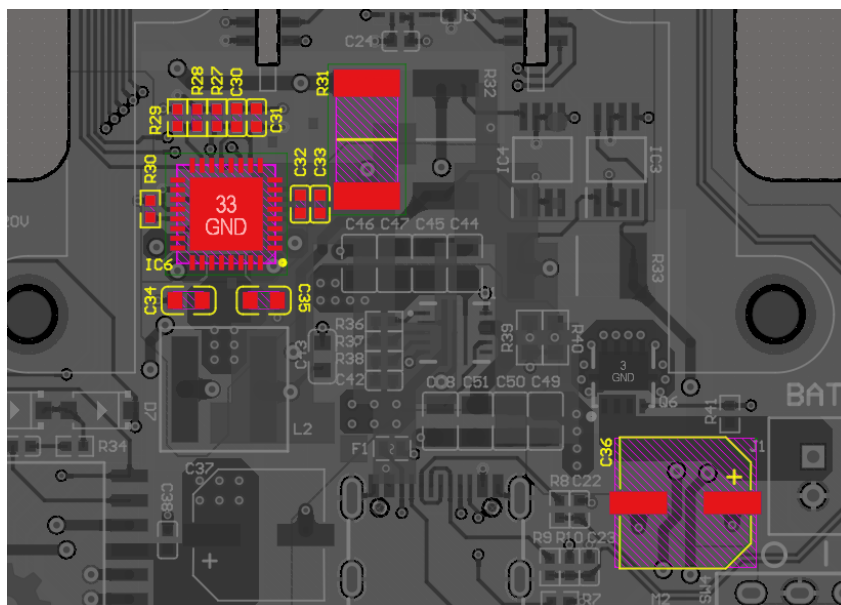


Figure 21 – Fan driver PCB.

5.2.8.5 Connectivity and Interface

The Human-Machine Interface was miniaturized and now consists of two smaller addressable RGB LEDs, a buzzer, a 4-way DIP switch, and three miniature tactile buttons: one general-purpose user button, one Reset button, and a newly added Boot button. Additionally, two discrete LEDs are present: one connected directly to the 3.3V line (Power Indicator) and one Status LED controlled by the MCU.

A central innovation is the USB-C implementation, leveraging the dual-role capabilities of the STM32 pins PA11 and PA12:

- **Physical Connection:** PA11 is connected to the USB DN pin, and PA12 is connected to the USB DP pin.
- **Mode 1 - UART Communication (Default):** In the firmware, PA11 and PA12 are configured as UART_RX and UART_TX, respectively. The external Bluetooth module (based on HM-19 ([TECHNOLOGY, 2025](#))) was designed to match this pinout. When the module is plugged into the USB-C port, the robot communicates via UART automatically.
- **Mode 2 - DFU Flashing:** If the user connects the robot to a PC via USB and holds the Boot button during startup, the microcontroller detects the USB signal on PA11/PA12. The internal bootloader then enters DFU (Device Firmware Upgrade) mode, allowing code flashing without external programmers.

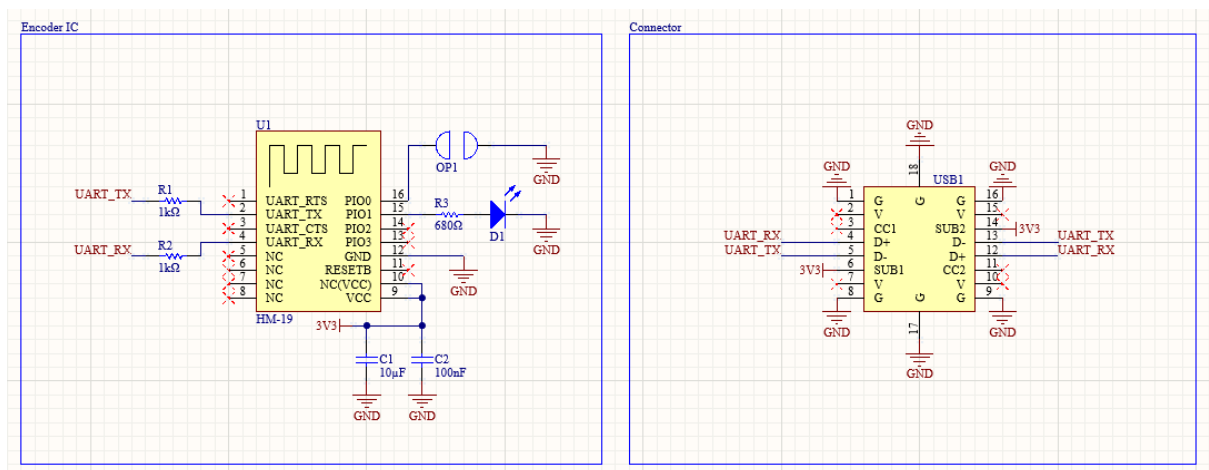


Figure 22 – Custom Bluetooth module schematics.

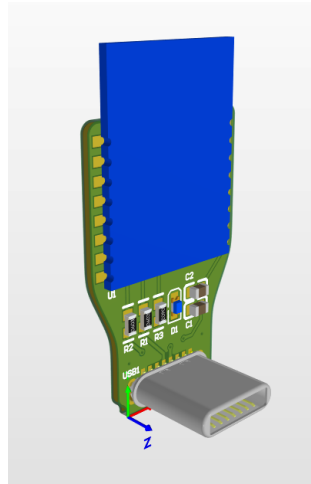


Figure 23 – Custom Bluetooth module PCB.

5.2.8.6 PCB Design and Signal Integrity

One of the major differentiators of this project lies in the advanced printed circuit board design. Due to the system's mixed nature, combining sensitive analog signals, fast digital buses (SPI, USB), and noisy power electronics, a 4-layer stackup (Signal - GND - Power - Signal) was adopted to ensure signal integrity (SI) and electromagnetic compatibility (EMC) (PETERSON, 2021).

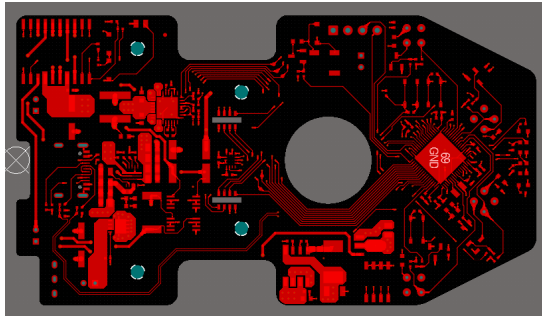
The main layout techniques applied include:

- **Single-Sided Component Placement:** To maximize the suction plenum volume and minimize aerodynamic turbulence, the Bottom Layer was kept entirely free of components. This required a high-density routing strategy to consolidate all circuitry on the Top Layer, successfully achieving this despite the 25% reduction in board area compared to the double-sided v1 design.
- **Solid Reference Plane (L2):** The second layer was dedicated exclusively to a continuous Ground (GND) plane. This provides a low-impedance return path immediately below the top layer signal traces, minimizing loop area and, consequently, electromagnetic interference (EMI) emission and susceptibility.
- **Split Power Planes (L3):** The third layer was used for power distribution, containing separate polygons for VBAT, 20V, 3.3V, and 5V.
- **Return Current Management:** Critical attention was given to traces routed on the bottom layer. To prevent signals from crossing "splits" between different voltage planes on L3, which would create large inductance loops, a layer hopping technique was used: the signal is brought to the top layer to cross the split over the solid GND

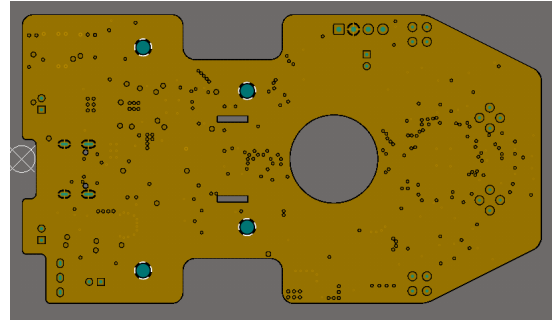
plane (L2) and then returned to the bottom layer, preserving the integrity of the return path (PETERSON, 2019).

- **Switching Loop Minimization:** The Buck and Boost regulator circuits, which operate at high switching frequencies, had their critical components (inductor, IC, and input/output capacitors) positioned and routed exclusively on the top layer with wide polygons, minimizing parasitic inductance.
- **USB Differential Routing:** The USB data lines (DP/DN) were routed as a differential pair with controlled 90-ohm impedance and length matching, ensuring robust and error-free data communication.

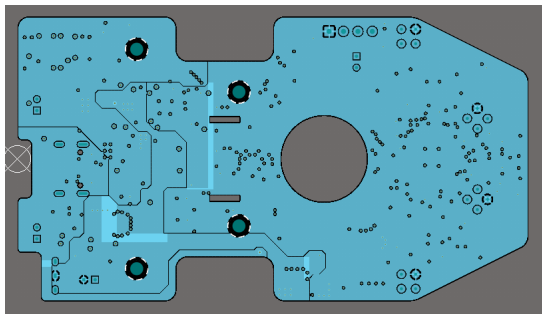
This rigorous approach to PCB design ensures that the physical hardware is not a limiting factor for navigation algorithms, providing clean sensor readings and precise, powerful motor actuation.



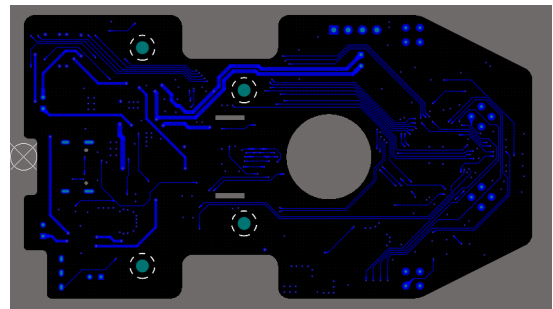
(a) Top Layer (L1).



(b) Ground Plane (L2).



(c) Power Plane (L3).



(d) Bottom Layer (L4).

Figure 24 – PCB Stackup (Rotated Views).

5.2.8.7 Manufacturing and Assembly

The physical implementation of the electrical architecture followed a hybrid manufacturing strategy, leveraging specific fabrication processes to balance cost, weight, and maintainability. The PCB (Printed Circuit Board) fabrication was outsourced using FR-4 Standard (TG135) substrate with a 4-layer stackup and a reduced thickness of 1.0 mm, contributing to the robot's overall mass reduction.

For the surface finish, HASL with lead (Hot Air Solder Leveling) was selected. This choice, over lead-free or ENIG finishes, facilitates manual soldering due to the lower melting point and superior wetting properties of leaded solder, which is advantageous for reworking and fixing complex components. Vias were specified as tented (covered with solder mask) to ensure electrical insulation, preventing accidental solder bridging and protecting the copper from oxidation. The copper weight distribution was set to 1 oz for outer layers and 0.5 oz for inner layers.

Due to the high component density and the extensive use of 0402 passive components, the automated assembly service (PCBA) was utilized for these parts, ensuring precision and significantly reducing bench time.

However, critical semiconductor components, specifically the STM32H725RGV6 microcontroller, the L6205D and STSPIN958 drivers, and the Buck regulator, were excluded from the automated process as they were acquired through STMicroelectronics' free sample program. Other components, such as sensors and encoders, were repurposed from the previous robot's inventory. Consequently, these ICs were soldered manually by the author, requiring advanced techniques to ensure proper thermal connection of the bottom pads on the drivers and the microcontroller.

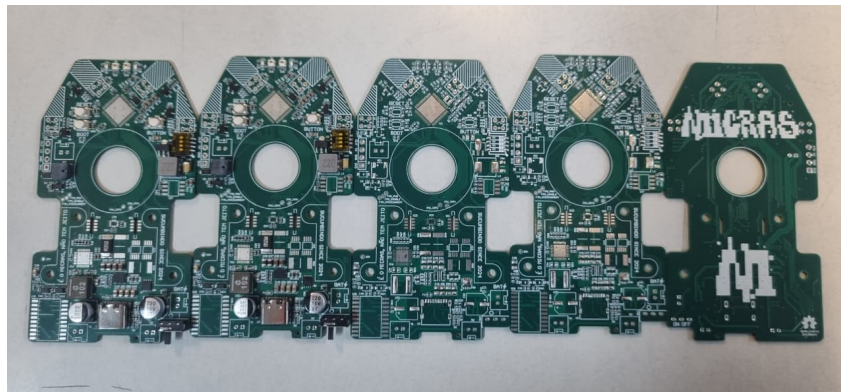
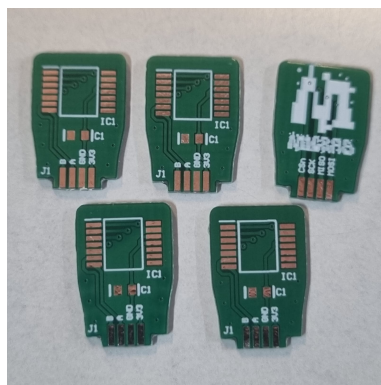
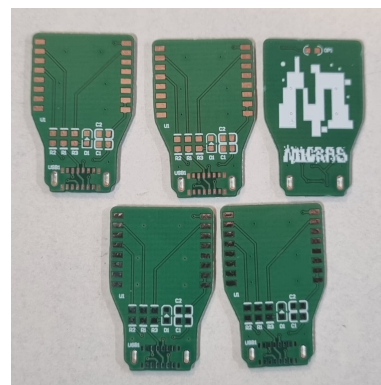


Figure 25 – Main Board batch: PCBA (left) and bare PCBs (right).



(a) Magnetic Encoder board PCB.



(b) Bluetooth Low Energy interface module.

Figure 26 – Auxiliary printed circuit boards designed for peripheral integration.

Finally, consistent with the open-source philosophy adopted for the mechanical design, the complete electronic engineering repository is publicly available. The native Altium Designer project files, schematics, and manufacturing outputs are hosted on the [GitHub repository](#) (SANTI, 2025). Furthermore, to facilitate design inspection without requiring specific CAD software, an interactive web-based visualization (powered by Altium 365) is accessible at [the project's website](#) (SANTI, 2025).

5.3 Computational Implementation

The firmware was developed in Bare Metal C++, leveraging CMake for the build system and the arm-none-eabi-g++ compiler. In adherence to the open-source philosophy of this project, the complete source code is publicly available in the [GitHub repository](#) (BARBOSA; SANTI; PEREIRA, 2025). The decision to forego a Real-Time Operating System (RTOS) was driven by the aim to maintain architectural simplicity. As the system's requirements did not necessitate complex task scheduling, a bare-metal approach facilitated a clearer understanding of the execution cycle and streamlined the debugging process.

A critical factor in the success of this iteration was the reuse of the layered software architecture established in the project's first version (Micras v1). This robust architecture isolates navigation logic from hardware specifics through strict abstraction levels:

- **STM32 HAL:** The manufacturer-provided layer (generated via STM32CubeMX), handling direct register access.
- **micras_hal:** A modern C++ wrapper encapsulating the ST HAL, providing a safe and efficient object-oriented interface.
- **micras_proxy:** An implementation of the Proxy design pattern (DOUGLASS, 2011), abstracting physical components (motors, sensors, buttons) into logical interfaces.

Due to this decoupling, where each layer accesses only the one immediately below it, migrating to the new STM32H725 microcontroller required only specific reconfigurations in STM32CubeMX and minor adjustments to the `micras_hal` layer. The high-level code remained hardware-agnostic. Furthermore, this structure enabled algorithm validation in a simulated environment by simply replacing the `micras_proxy` layer with software mocks, without altering the control logic.

With the low-level foundation stabilized, development efforts focused on restructuring the decision-making and navigation logic.

5.3.1 Top Level Logic and Finite State Machine

The software development of the Micras robot underwent a significant structural transformation to meet the reliability and scalability requirements of high-performance Micromouse competitions. The primary objective of this refactoring was to transition from a monolithic, procedural control loop to a modular, event-driven architecture.

This section details the high-level architectural design, focusing on the decision-making core (Finite State Machine) and the Human-Machine Interface (HMI) abstraction.

5.3.1.1 Architectural Paradigm Shift

In previous iterations, the robot's behavior was governed by a centralized "Super Loop" controller. While simple to implement initially, this approach coupled low-level hardware management (e.g., reading raw sensor values) directly with high-level logic (e.g., deciding whether to explore or solve). As the complexity of the navigation algorithms increased, this coupling resulted in a rigid system where adding new features risk destabilizing existing functionality.

The new architecture adopts an Object-Oriented design, specifically utilizing the State Pattern ([GAMMA et al., 1994](#)). This approach separates the robot's logic into distinct, self-contained modules. The central execution flow is no longer a series of conditional checks but a delegation of authority to the currently active "State" object. This decoupling ensures that the navigation logic does not need to know the specifics of the hardware implementation, and conversely, the hardware drivers operate independently of the current mission objective.

5.3.1.2 Finite State Machine (FSM)

At the core of the new system is a custom Finite State Machine (FSM). The FSM models the robot's lifecycle as a set of discrete states, where the robot can only exist in one state at a time. This model aligns perfectly with the Micromouse competition flow, which consists of distinct phases: calibration, configuration, exploration, and fast runs.

5.3.1.3 State definitions

The behavior is compartmentalized into the following primary states:

- **Initialization (Init):** The system performs hardware self-checks (IMU connectivity, sensor functionality) and initializes the communication peripherals.
- **Idle:** The default resting state. Here, the robot awaits user input. All motors are disabled, and the system consumes minimal power while polling the Interface layer for configuration commands.

- **Calibration:** A specialized state for sensor tuning. It isolates the logic required to normalize wall sensor readings and gyroscope offsets, ensuring these critical values are not affected by other running processes.
- **Run:** The active state where navigation algorithms take control. This state encapsulates both "Exploration" (mapping the maze) and "Solving" (executing the fastest path).
- **Error:** A safety trap state. If a critical hardware failure or a software exception (e.g., crash detection) occurs, the system locks into this state to prevent hardware damage, signaling the error via the HMI.

5.3.1.4 State Transitions and Flow

Transitions between these states are deterministic and triggered by specific events processed by the FSM. The diagram below illustrates the high-level flow of the system:

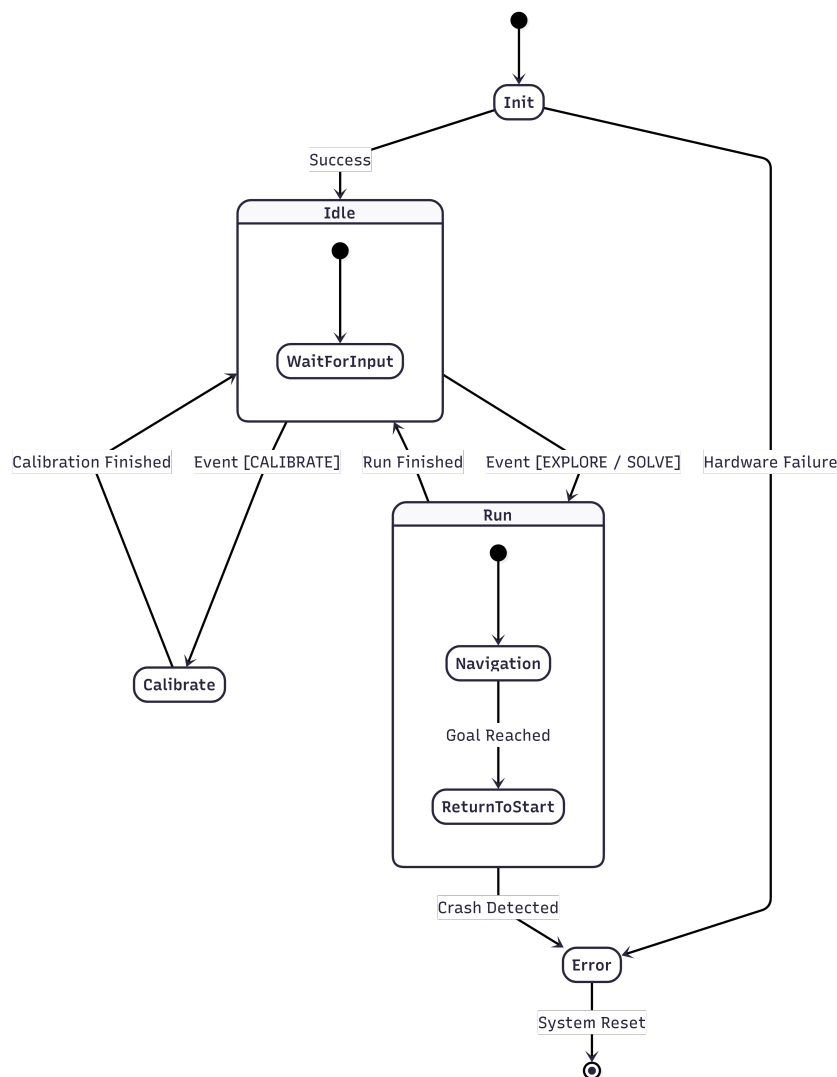


Figure 27 – Robot's main FSM.

By encapsulating the logic within these states, the system achieves a high degree of testability and modularity.

5.3.1.5 Human-Machine Interface (HMI) Abstraction

A critical aspect of a competition robot is the efficiency of the interaction between the operator and the machine. During a competition, the operator must quickly configure the robot's strategy (e.g., switching from "Safe Exploration" to "Risky Speed Run") using limited physical inputs.

To manage this, the firmware introduces a dedicated Interface abstraction layer. This module acts as a bridge between the physical world (Switch, Buttons, LEDs) and the logical world (Events, Objectives).

5.3.1.6 Event-Driven Configuration

The **Interface** class aggregates the status of various hardware components and translates them into semantic Events. This abstraction relies on the underlying Proxy layer to handle low-level signal processing, such as button debouncing and sensor filtering. The Interface layer's primary responsibility is to map these pre-processed hardware conditions to high-level triggers.

The system maps inputs to events as follows:

- **Condition Check:** The Interface evaluates specific conditions defined by the hardware state (e.g., "Is the button in a 'Short Press' state?" or "Has a specific DIP switch been toggled?").
- **Event Generation:** If a condition is met, a semantic event is dispatched (e.g., `Event::EXPLORE`, `Event::SOLVE`). This allows the core logic to subscribe to "What happened" (User wants to Explore) rather than "How it happened" (Button Pin 3 went high).

5.3.1.7 Configuration Robustness

This layer also manages the robot's configuration profile via DIP switches. Before a run, the **Interface** reads the state of the switches to determine:

1. **Strategy:** Whether to prioritize diagonal paths or orthogonal-only movement.
2. **Performance:** Whether to enable the vacuum fan for increased traction.
3. **Risk Profile:** Whether to execute a "Risky" run (higher speeds, lower safety margins) or a "Conservative" run.

By abstracting these inputs, the architecture allows for Virtual Control. Since the FSM listens for standardized Events, the `micras_comm` telemetry module can inject these same events remotely via Bluetooth. This means the robot can be configured and started wirelessly from a computer using the exact same logic path as the physical buttons, significantly simplifying the debugging and testing process.

5.3.1.8 Communication and Telemetry System (`micras_comm`)

To address the challenges of debugging and tuning in a real-time embedded environment, a dedicated communication module, `micras_comm`, was developed. This system provides comprehensive system observability and enables runtime parameter adjustment, eliminating the need for repetitive recompilation cycles for fine-tuning control constants.

The implementation relies on the interaction between three core classes: `Packet`, which defines the data structure; `SerialVariablePool`, which manages the registry of observable variables; and `CommunicationService`, which orchestrates the data flow.

5.3.1.8.1 Packet Structure and Protocol

To ensure data integrity over potentially noisy wireless links (Bluetooth/UART), a custom packet-based binary protocol was implemented in the `Packet` class. Each transmission follows a strict frame structure initiated by a fixed Header byte (`0x42`) and terminated by a Tail byte (`0x7F`). The payload encapsulates the Message Type, a 16-bit Variable ID, the Data Size, and the Data itself.

To prevent framing errors, the class implements a Byte Stuffing mechanism. The `escape_payload` method iterates through the data stream, and any reserved byte (Header, Tail, or the Escape byte itself) is replaced by an escape sequence. This ensures that the frame delimiters remain unique, allowing the parser to reliably recover synchronization even in the event of data loss. A checksum is appended to every packet to validate integrity before processing.

5.3.1.8.2 Serial Variable Pool

The `SerialVariablePool` class acts as a dynamic registry where internal C++ variables are "published". Instead of hard-coding memory addresses, parameters such as PID gains or sensor readings are mapped to unique 16-bit IDs.

To allow the client to correctly interpret these IDs, the system implements a Handshake Mechanism. Upon connection, the external client sends a Map Request, and the robot responds (via `serialize_var_map`) with a metadata catalog containing the ID,

Name, Data Type, and Access Rights for every registered variable. This decouples the firmware from the client interface, allowing generic dashboards to interact with the robot without prior knowledge of the specific firmware version.

5.3.1.8.3 Service Execution Model

The `CommunicationService` class orchestrates the reception and transmission logic. Designed to operate within the main Bare Metal loop, it employs a Hybrid Polling architecture implemented in the `update()` method:

1. **Non-Blocking Ingestion:** The service fetches raw bytes from the hardware buffer.
2. **Deframing and Queueing:** Valid packets are reconstructed by the `Packet` class and pushed into an internal FIFO queue (`incoming_packets`).
3. **Synchronous Processing:** The service consumes the queued packets sequentially within the same cycle, executing commands such as `SERIAL_VARIABLE` updates or responding to Pings.

This design ensures that telemetry data is processed as fast as the main loop allows, while maintaining the determinism required for the robot's control system.

5.3.2 Navigation System Architecture

The firmware is built upon a modular architecture, with the `micras_nav` module serving as the core decision-making unit. This module is responsible for localization, mapping, path planning, and motion control. The system operates on a continuous loop where sensor data is processed to update the robot's state, which in turn informs the high-level planner and low-level controllers.

The architecture is divided into several specialized components:

- **Odometry:** Responsible for estimating the robot's pose (position and orientation) and velocity.
- **Costmap:** Maintains a grid-based representation of the maze costs using propagation algorithms.
- **Maze Logic:** Manages the state of walls, visited cells, and high-level decision making (Exploration vs. Solving).
- **Action Queuer:** Translates abstract grid paths into optimized, physics-compliant motion primitives.

- **Speed Controller:** Executes the motion commands using feedback and feed-forward control loops.
- **Follow Wall:** Provides active correction to keep the robot centered in the maze corridors.

5.3.3 Localization and Mapping

5.3.3.1 Odometry

The `Odometry` class estimates the robot's state $\mathbf{x} = [x, y, \theta, v, \omega]^T$ by integrating sensor data. It utilizes two rotary encoders (one on each wheel) and an Inertial Measurement Unit (IMU).

To minimize noise from the mechanical encoders, a Butterworth filter is applied to the raw velocity measurements before integration. The position is updated continuously using the kinematic model of a differential drive robot:

$$\begin{aligned}x_{t+1} &= x_t + v \cdot \cos(\theta) \cdot \Delta t \\y_{t+1} &= y_t + v \cdot \sin(\theta) \cdot \Delta t \\\theta_{t+1} &= \theta_t + \omega \cdot \Delta t\end{aligned}$$

Where v is the linear velocity derived from the encoders and ω is the angular velocity obtained from the IMU gyroscope.

Initially, the system relied on a global odometry approach, where the robot continuously tracked its absolute (x, y) coordinates within the maze. However, this method suffered from unbounded error accumulation over time, necessitating complex logic to identify "known states" for resetting the global position error.

To address this, the architecture was shifted to a Local Odometry model. In this paradigm, the odometry is reset at the beginning of each discrete action (e.g., moving forward one cell). The robot relies on the relative distance traveled to determine when an action is complete, rather than checking absolute coordinates.

This approach isolates position errors to individual segments. Angular errors are actively corrected during the motion using the `FollowWall` logic (when side walls are present). If an action ends with a residual angular error and the subsequent segment lacks side walls for correction, the robot can utilize the front wall of the upcoming cell for alignment. This strategy significantly reduces the complexity of long-term error management.

5.3.3.2 Costmap and Flood Fill

The maze is represented as a 16×16 grid. The **Costmap** class implements a modified Flood Fill algorithm (LAW, 2013) to calculate the minimum distance (cost) from every cell to a specific target. This algorithm propagates costs outwards from the target cell (cost 0) to all reachable neighbors (cost $n + 1$), respecting the known walls.

The system maintains two independent cost layers:

1. Explore Layer: Calculates costs to the center of the maze (the goal).
2. Return Layer: Calculates costs back to the starting cell (0,0).

This dual-layer approach allows the robot to instantly switch between "going to goal" and "returning home" behaviors without recomputing the entire map.

5.3.4 Exploration and Path Planning

The **Maze** class orchestrates the robot's strategy. It handles wall updates from sensor observations and decides the next target cell.

5.3.4.1 Wall Detection and Updates

As the robot moves, the distance sensors (Left, Front Left, Front Right, Right) detect walls. When a new wall is discovered, the **Costmap** is locally updated, and the Flood Fill algorithm re-propagates costs only if necessary. This ensures the map always reflects the current knowledge of the environment.

5.3.4.2 Exploration Strategy (BFS)

During the exploration phase, the robot must visit unknown areas. The algorithm uses a Breadth-First Search (BFS) to find the nearest "interesting" cell. An interesting cell is defined as a reachable cell that has at least one unobserved wall.

$$\text{Next Goal} = \min_{\text{cost}} \{c \in \text{Grid} \mid \text{is_reachable}(c) \wedge \text{has_unknown_walls}(c)\}$$

This ensures a systematic exploration that minimizes redundant travel.

5.3.4.3 Dead End Optimization

To optimize exploration, the system detects "dead ends" (cells with 3 walls). When the robot enters a dead end, it marks the entrance with a virtual wall. This effectively

seals off the dead end in the internal map, preventing the path planner from ever entering that cell again, significantly reducing search time in subsequent runs.

5.3.4.4 Fastest Route Solver (DFS)

Once the maze is explored, the robot must compute the fastest path to the goal. Unlike the exploration phase which minimizes distance (cells), the solving phase minimizes time.

The system employs a Depth-First Search (DFS) with Recursive Backtracking.

1. **Search:** It recursively explores all possible loop-free paths from start to goal.
2. **Heuristic Pruning:** It uses the Manhattan distance ([RUSSELL; NORVIG, 2010](#)) as a heuristic. If the current path length plus the heuristic estimate exceeds the best time found so far, the branch is pruned.
3. **Time Estimation:** For every candidate path, the algorithm simulates the robot's physics to calculate the theoretical travel time, considering acceleration limits and curve speeds.

This approach is superior to A* or Dijkstra for this specific application because the "cost" of a path is not a simple sum of edge weights (due to the robot's momentum and continuous movement), but a complex function of the trajectory's geometry.

5.3.5 Motion Planning (Action Queuer)

The `ActionQueuer` is the most sophisticated component of the navigation stack. It bridges the gap between the discrete grid path (e.g., $(0,0) \rightarrow (0,1) \rightarrow (1,1)$) and the continuous control signals required by the motors.

5.3.5.1 Motion Primitives

The path is first decomposed into a sequence of atomic actions:

- **START:** Alignment and initial acceleration.
- **MOVE_FORWARD:** Linear movement of a specific distance.
- **TURN:** In-motion rotation (90 degrees).
- **DIAGONAL:** Combined linear and angular movement (45 degrees).
- **SPIN:** In-place rotation (for dead ends).
- **STOP:** Deceleration to zero velocity.

5.3.5.2 Trajectory Optimization Pipeline

The raw sequence of actions undergoes several optimization passes to maximize speed and smoothness.

1. **Diagonalization:** The planner scans the action list for "L-shaped" patterns (e.g., Turn Left $\rightarrow$ Move $\rightarrow$ Turn Right). These are replaced by Diagonal actions. A diagonal movement cuts across the grid at 45° , reducing the total distance traveled by factor of $\sqrt{2}$ and, more importantly, reducing the total rotation angle required.
2. **Action Fusion:** Consecutive `MOVE_FORWARD` commands are merged into single, longer movements. This reduces the processing overhead and allows the speed controller to accelerate to higher top speeds over long straight sections.
3. **Continuous Turns and Trimming:** To avoid stopping at every corner, the robot executes continuous turns. The planner calculates a "trim distance" for every turn.

Trim Before: The distance before the corner where the robot must start turning.

Trim After: The distance after the corner where the turn completes.

These distances are subtracted from the adjacent linear movements. This effectively "cuts the corners" of the grid, creating a smooth, arc-like trajectory that maintains linear momentum.

5.3.5.3 Dynamic Action Compilation

Finally, the optimized abstract actions are compiled into physics-based `MoveAction` and `TurnAction` objects.

- **MoveAction:** Uses the Torricelli equation ($v^2 = v_0^2 + 2a\Delta s$) to calculate the required acceleration or deceleration profile to reach the target speed at the end of the segment.
- **TurnAction:** Uses a triangular angular velocity profile to execute the turn while maintaining the linear velocity calculated by the curve profiling logic.

5.3.6 Trajectory Generation and Curve Profiling

To execute turns at high speeds without losing traction or stability, it is necessary to determine the maximum permissible angular velocity that results in the correct final heading and position. Unlike simple circular arcs, the robot's angular velocity is not constant; it follows a trapezoidal profile to respect the angular acceleration limits of the physical platform.

5.3.6.1 Mathematical Derivation of Curve Dynamics

The motion of the robot during a curve can be modeled starting from the kinematic equation for displacement in the y -axis (lateral displacement relative to the curve start):

$$dy = v_l \cdot \sin(\theta) \cdot dt$$

Integrating both sides gives the total lateral displacement y :

$$y = \int v_l \cdot \sin(\theta) dt$$

Assuming the linear velocity v_l remains constant throughout the curve (a key feature of our motion planner), it can be moved outside the integral. Geometrically, a perfect curve with radius r and total angle θ_{total} results in a lateral displacement of $r \cdot (1 - \cos(\theta_{total}))$. Thus, we establish the fundamental constraint:

$$\int_0^T \sin(\theta) dt = \frac{r \cdot (1 - \cos(\theta_{total}))}{v_l}$$

Here, T is the total duration of the curve, and $\theta(t)$ is the instantaneous orientation of the robot. To solve this, we must define $\theta(t)$ based on the angular velocity profile.

5.3.6.2 Trapezoidal Angular Velocity Profile

The angular velocity $\omega(t)$ follows a trapezoidal profile: it accelerates from 0 to ω_{max} at a rate α_{max} , holds ω_{max} , and then decelerates back to 0. This divides the curve into three time intervals: $[t_0, t_1]$ (acceleration), $[t_1, t_2]$ (constant velocity), and $[t_2, t_3]$ (deceleration).

The orientation $\theta(t)$ in each interval is obtained by integrating $\omega(t)$:

1. Acceleration Phase: $\theta(t) = \frac{\alpha_{max} \cdot t^2}{2}$
2. Constant Phase: $\theta(t) = \theta_1 + \omega_{max} \cdot t$
3. Deceleration Phase: $\theta(t) = \theta_2 + \omega_{max} \cdot t - \frac{\alpha_{max} \cdot t^2}{2}$

The duration of the acceleration and deceleration phases is determined by the time required to reach ω_{max} : $t_{accel} = \frac{\omega_{max}}{\alpha_{max}}$.

The angular displacement during acceleration (and deceleration, by symmetry) is:

$$\theta_{accel} = \frac{\omega_{max}^2}{2\alpha_{max}}$$

The remaining angle $\theta_{const} = \theta_{total} - 2\theta_{accel}$ is covered during the constant velocity phase.

5.3.6.3 Numerical Approximation for Real-time Calculation

Substituting these expressions back into the main integral yields a sum of integrals of sine functions with quadratic arguments (Fresnel-like integrals), which do not have a closed-form analytical solution suitable for embedded execution.

$$\int_0^T \sin(\theta) dt = \int_{accel} \dots + \int_{const} \dots + \int_{decel} \dots$$

To solve this on the microcontroller, the relationship between the maximum angular velocity ω_{max} and the curve parameters was analyzed numerically. By plotting the value of the integral against ω_{max} , a behavior closely resembling a function of the form $f(x) = \frac{1}{x} + cx$ was identified.

This observation allows the approximation of the complex integral constraint with a much simpler algebraic equation:

$$\frac{1}{\omega_{max}} + \frac{\omega_{max}}{k \cdot \alpha_{max}} = \frac{r \cdot (1 - \cos(\theta_{total}))}{v_l}$$

Where k is a correction factor derived from regression analysis of the numerical solution.

5.3.6.4 Solution

This approximation reduces the problem to a quadratic equation in terms of ω_{max} . The firmware solves this equation in real-time using the Bhaskara formula to determine the exact maximum angular velocity required for any given curve radius, linear velocity, and total angle.

$$\omega_{max} = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

```

1 /**
2  * @brief Calculate the maximum angular speed for a given curve
3     radius and linear speed.
4  *
5  * @param angle Angle to turn in radians.
6  * @param curve_radius Radius of the curve in meters.
7  * @param max_angular_acceleration Maximum angular acceleration
8     in rad/s^2.

```

```

7  * @param max_centripetal_acceleration Maximum centripetal
   * acceleration in m/s^2.
8  * @return Maximum angular speed in rad/s.
9  *
10 * @details Maximum angular velocity is computed to generate a
   * curve equivalent in displacement to one that
11 * maintains the desired radius of curvature throughout the
   * entire turn.
12 * If linear speed is zero, a minimal angular speed is assigned.
13 */
14 static constexpr float calculate_max_angular_speed(
15     float angle, float curve_radius, float
16     max_angular_acceleration, float
17     max_centripetal_acceleration
18 ) {
19     if (curve_radius == 0.0F) {
20         return max_angular_acceleration * 0.01F;
21     }
22
23     const float correction_factor =
24         14.35F - 13.57F * std::cos(std::abs(angle) - 2) - 10.0F /
25         max_angular_acceleration;
26
27     const float transformed_acceleration =
28         std::pow(1.0F - std::cos(angle), 2.0F) *
29         correction_factor * max_angular_acceleration;
30
31     const float max_angular_speed = std::sqrt(
32         (max_centripetal_acceleration * transformed_acceleration)
33         /
34         (curve_radius * transformed_acceleration -
35         max_centripetal_acceleration)
36     );
37
38     if (std::isnan(max_angular_speed)) {
39         std::terminate();
40     }
41
42     return max_angular_speed;
43 }

```

Listing 5.1 – Calculation Function for Maximum Angular Velocity

This allows the robot to dynamically adapt its turning speed to different maze geometries while guaranteeing that the physical constraints of angular acceleration and linear velocity are met, resulting in precise and repeatable trajectories.

5.3.7 Control Systems

5.3.7.1 Speed Controller

A major aspect of the system refactoring was the migration from Position Control to Velocity Control.

Previously, the robot primarily performed control over its position. A target point was compared to the robot's current position to generate commands. While this guaranteed reaching the destination, it lacked consistency in execution: any change in control constants affected the robot's speed profile along the path.

An alternative considered was to pre-define the velocity profile. This guarantees consistency and smoother curves but, in an open-loop scenario, offers no feedback for deviations. Adding a closed-loop velocity controller introduces a new challenge: it typically requires a significant integral term to reach the setpoint, causing system lag.

The Chosen Solution: Velocity Control with Feed-Forward

The system now implements a velocity controller augmented with a feed-forward model. By modeling the system physics, the voltage required to achieve a target velocity (V_{ff}) is estimated. The PID controller then operates around this value to correct small errors. This approach significantly improves response time, as the feed-forward term provides the bulk of the control effort, reducing the reliance on the integral term.

This shift allows for:

- **Consistency:** The robot follows the planned velocity profile regardless of battery voltage or minor friction changes.
- **Smoothness:** Curves are executed with optimal velocity profiles.
- **Responsiveness:** The feed-forward term eliminates the lag typical of pure velocity PIDs.

The `SpeedController` ensures the robot follows the generated velocity profiles. It employs a PID controller augmented with a Feed-Forward term.

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} + F(v_{target}, a_{target})$$

The Feed-Forward term F predicts the voltage required to maintain the target velocity and acceleration based on the motor's physical model (Back-EMF and inertia). This allows the PID controller to focus only on correcting small disturbances, resulting in much faster response times.

5.3.7.2 Wall Following and Odometry Correction

Over long distances, wheel slippage and small angular errors can cause the robot to drift. The `FollowWall` class implements an active correction system that uses the infrared distance sensors to keep the robot centered in the corridor and longitudinally calibrated.

- Lateral Correction

The system uses a PID controller to adjust the robot's angular velocity based on the distance to the side walls.

- Two Walls: If both walls are present, the error is the difference between the left and right sensor errors ($e = e_{left} - e_{right}$), keeping the robot equidistant.
- One Wall: If only one wall is visible, the error is derived from that single sensor relative to a reference distance.

The output of this PID is added to the angular velocity command, effectively "steering" the robot back to the center line.

- Longitudinal Correction (Post Detection)

A critical feature for maintaining accurate odometry is the detection of "posts", which the border of each cell's wall. As the robot passes a wall opening, the sensor reading changes rapidly. The system calculates the spatial derivative of the sensor reading:

$$\frac{d(\text{reading})}{d(\text{distance})} \approx \frac{\Delta \text{reading}}{\Delta \text{distance}}$$

When this derivative exceeds a calibrated threshold, a post is detected. Since the position of posts in the grid is known and fixed (at the edges of the 180mm cells), the robot uses this event to reset its longitudinal position error. This periodic recalibration virtually eliminates cumulative odometry errors in the direction of travel, allowing the robot to navigate indefinitely without losing track of its position within the cell grid.

5.4 Tests and Evaluation

Given the project's aggressive timeline and the complexity of the redesign, the validation strategy adopted a multi-stage approach. Instead of relying solely on final field tests, each subsystem (Software, Electronics, and Mechanics) underwent rigorous verification using simulation tools, Design Rule Checks (DRC), and physical prototyping prior to the final integration.

5.4.1 Computational Validation: Simulation and Legacy Hardware

The navigation architecture, specifically the `micras_nav` module, was validated primarily through a custom simulation environment developed in C++. This approach allowed for the testing of high-level algorithms, such as the `ActionQueuer`, Flood Fill mapping, and Feed-Forward control logic, decoupled from the physical hardware availability.

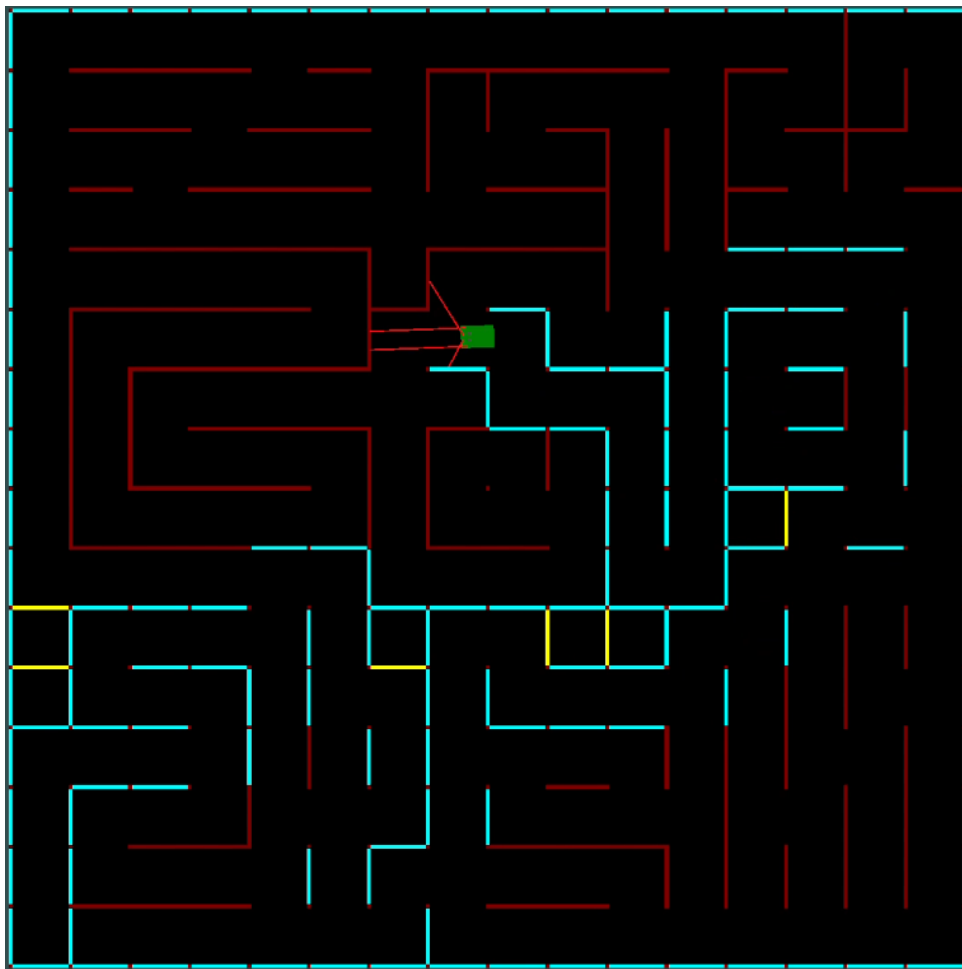


Figure 28 – Simulation environment: Robot executing the exploration algorithm and mapping the maze. The mapped walls are highlighted in blue, and the virtual walls in yellow

The simulation confirmed the robot's ability to:

- Perform efficient exploration using the weighted BFS algorithm.
- Calculate optimal speed runs, successfully executing diagonal movements and smooth curves without stopping, validating the trajectory planning logic.
- Recover from localized odometry errors using the simulated wall sensors.

Crucially, the new high-level navigation software was also validated on the Micras v1 hardware. Thanks to the robust layered architecture established in the previous version, no back-porting or code modification was required. The hardware abstraction layers (`micras_hal` and `micras_proxy`) successfully isolated the logic from the underlying hardware differences (STM32G4 vs. STM32H7). This allowed the exact same navigation stack designed for the v2 to run on the v1, validating the modularity of the firmware and the state machine logic in a physical embedded environment, despite the mechanical limitations of the older robot.

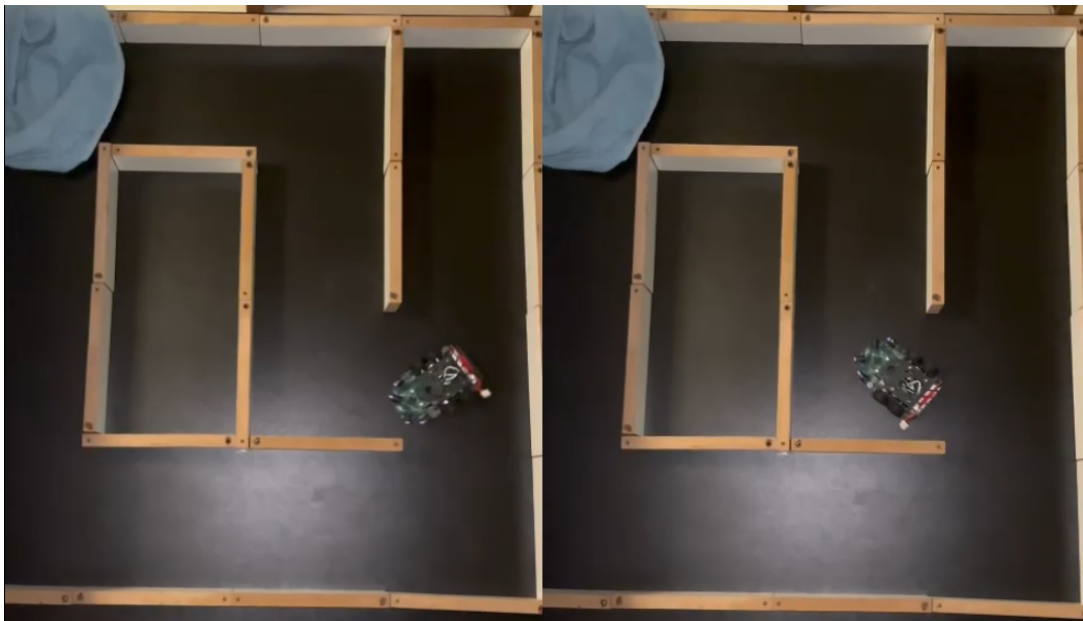


Figure 29 – Validation of the new software architecture running on the previous version hardware.

5.4.2 Electronic Validation: Manufacturing and Signal Integrity

The electronic validation focused on ensuring design manufacturability and signal integrity, given the high density of the 4-layer PCB.

- **Design Validation:** The PCB design underwent strict Design Rule Checks (DRC) within Altium Designer to ensure compliance with clearance, track width, and via

constraints. Furthermore, the design passed the Design For Manufacturing (DFM) checks performed by the manufacturer (JLCPCB) without the need for revisions, a significant improvement over the previous version, which required multiple design adjustments before fabrication.

- **Physical Verification:** Upon receipt of the manufactured boards, the manual assembly of critical components (STM32H7, Drivers, Regulators) was followed by electrical testing:
- **Continuity and Short-Circuit Testing:** Verified using a multimeter to ensure no solder bridges existed on the fine-pitch components, particularly the VFQFPN68 microcontroller.
- **Signal Verification:** An oscilloscope was used to validate the stability of the power rails (3.3V and 20V Boost) and the signal integrity of the programming interface (SWD) prior to the first firmware upload.

5.4.3 Mechanical Validation: Fit and Function

The mechanical redesign was validated through CAD interference analysis and rapid prototyping.

- **Virtual Assembly:** The SolidWorks assembly was analyzed to ensure the center of mass was lowered and centered relative to the wheel axis.
- **Physical Prototyping:** The chassis and motor mounts were manufactured using SLA (Resin) 3D printing. A physical "fit check" was performed using the printed parts and the manufactured PCB. This confirmed the precise interference fit of the bearings and the alignment of the perpendicular encoder boards, validating the mechanical tolerances.

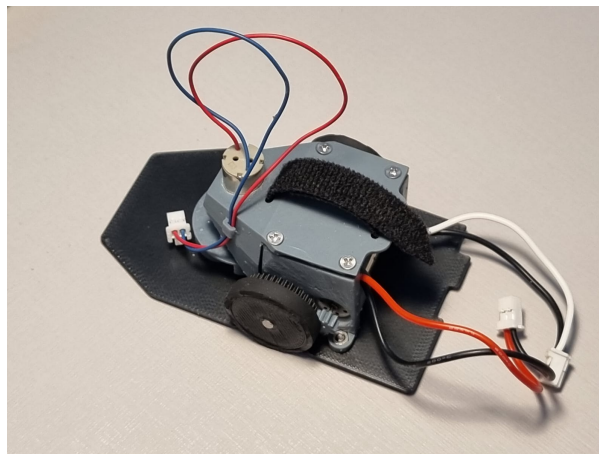


Figure 30 – Mechanical validation: 3D printed prototype assembly and PCB fit check.

5.4.4 Comparative Analysis: Micras v1 vs. Micras v2

The engineering efforts in this iteration resulted in quantifiable improvements in the robot's physical and computational specifications. The table below summarizes the key technical advancements achieved in the Micras v2 compared to its predecessor.

Parameter	Micras v1	Micras v2	Improvement
Microcontroller	STM32G4 (Cortex-M4)	STM32H7 (Cortex-M7)	$\approx 3.2\times$ Clock Speed
System Footprint	67×100 mm	50×90 mm	$\approx 33\%$ Reduction
Power System	2S (7.4V) / 12V Boost	3S (11.1V) / 20V Boost	Higher Torque/Efficiency
Locomotion	4 Wheels (AWD)	2 Wheels (Differential)	Reduced Inertia/Friction

Table 1 – Comparative analysis between the first and second versions of the robot.

The upgrade to the Cortex-M7 core (550 MHz) significantly enhances the system's real-time performance. While the fundamental algorithms could theoretically run on the previous architecture, the increased clock speed drastically reduces the control loop cycle time. This provides substantial computational headroom, ensuring that complex floating-point calculations for Feed-Forward control and trajectory optimization are executed with a high safety margin, preventing timing violations even under peak processing loads. Furthermore, the 33% reduction in the total system footprint, achieved through dense PCB routing and strategic mechanical cutouts for the wheels, directly contributes to a lighter robot with lower rotational inertia, enhancing dynamic performance in turns.

6 Final Considerations

This chapter presents the conclusions regarding the development of the Micras v2 project, summarizing the results achieved against the proposed objectives, the contributions made to the academic repository, and the perspectives for future work.

6.1 Conclusions

The primary objective of this work, the complete redesign of a Micromouse autonomous robot to overcome the mechanical and computational limitations of the previous iteration, was successfully achieved from an engineering perspective. This project represented a significant technical challenge, as it involved the individual execution of mechanical, electrical, and software development cycles within a compressed timeframe of approximately three months, in contrast to the typical one-year cycle performed by teams.

The development of Micras v2 directly addressed and resolved the critical bottlenecks identified in the "Future Works" of the Micras v1 thesis:

- **Computational Power:** The migration from the STM32G4 to the STM32H725 (Cortex-M7) provided the necessary processing headroom to implement the proposed feed-forward control and trajectory optimization algorithms, which were previously impossible due to hardware limitations.
- **Mechanical Stability:** The shift to a differential drive system with a lowered center of mass and increased gear reduction ratio (3.8:1) effectively solved the torque and backlash issues that compromised the previous four-wheel design.
- **Electrical Robustness:** Although the previous version also utilized a 4-layer PCB, the V2 implemented a rigorously optimized stackup configuration (*Signal - GND - Power - Signal*) focused on Signal Integrity (SI) and Electromagnetic Compatibility (EMC). By enforcing a solid ground reference plane (L2) and managing return current paths to avoid split crossings, the new design significantly reduced noise susceptibility. This, combined with the integration of a high-power 3S battery system and dedicated drivers, resolved the thermal shutdown issues and ensured stable operation under high-frequency switching.

Regarding the validation, while the complete autonomous maze solving in a physical environment was constrained by supply chain delays and the logistical complexity of manually assembling fine-pitch components (provided as samples), the system was

rigorously validated in stages. The mechanical fit was confirmed via prototyping, the electrical integrity was verified through laboratory testing, and the navigation logic was exhaustively proven in a custom simulation environment.

Therefore, it is concluded that the Micras v2 establishes a vastly superior platform compared to its predecessor, possessing the hardware capabilities and software architecture necessary to compete at a high level.

6.1.1 Contributions

Beyond the specific engineering solutions developed for the robot, this work aims to provide lasting value to the academic and competitive robotics communities. The main contributions include:

- **Open-Source Reference Architecture for the Brazilian Community:** Recognizing that the Micromouse category is still in its nascent stages in Brazil, this project releases its entire engineering repository as Open Source. By making the Altium Designer PCB projects, SolidWorks CAD models, and the C++ firmware fully available, this work serves as a high-performance reference design. This initiative aims to lower the barrier to entry for new teams and foster the technical growth of the national competitive robotics ecosystem.
- **micras_nav Module:** A decoupled, portable C++ navigation library that implements local odometry, flood-fill mapping, and profile generation. Its modular architecture allows it to be ported to other mobile robots with minimal adaptation, serving as a didactic resource for students learning about autonomous navigation and software architecture in embedded systems.
- **Mathematical Model for Trapezoidal Curves:** The derivation and numerical approximation developed to calculate the maximum angular velocity for non-circular curves provide a robust method for trajectory planning. This theoretical contribution addresses a common challenge in differential drive robots with acceleration constraints, offering a solution that balances mathematical precision with real-time execution capabilities.

6.1.2 Prospects for Continuity

Although the engineering phase establishes a solid platform, the iterative nature of robotics development reveals several opportunities for optimization. Based on the validation process and the identified limitations, the following steps are suggested for future work:

6.1.2.1 Short-Term Integration and Validation

Before pursuing hardware upgrades, the immediate priority is the conclusion of the physical assembly and the holistic validation of the platform.

- **Full System Commissioning:** Finalization of the hardware assembly procedures and the definitive integration of the firmware stack with the physical sensors and actuators. This step ensures that all electromechanical interfaces are operating within nominal parameters before dynamic testing.
- **Dynamic Tuning in Physical Environment:** Transferring the control parameters (PID/Feed-Forward) tuned in the simulation to the real robot. This step involves fine-tuning the constants to account for real-world friction, tire slippage, and sensor noise in a physical maze.

6.1.2.2 Electronics and Hardware

- **PCB Revision:** A minor revision of the PCB layout is required to adjust the placement of the USB-C connector. The current recessed position interferes with the housing of certain standard cables, making connection difficult.
- **Wireless Debugging Module:** Development of a hybrid embedded module combining a Debug Probe (SWD) and Bluetooth. This would enable cable-free flashing, debugging, and profiling, significantly speeding up the development cycle.
- **Custom Battery Pack:** The current commercial 3S battery is disproportionately large for the robot's form factor. Manufacturing a custom battery pack using smaller LiPo cells would improve the mechanical fit and allow for better mass centralization.

6.1.2.3 Mechanics and Actuation

- **Motor Upgrade:** The current power stage (20V Boost) was explicitly designed to support high-performance motors, such as the **Maxon DCX 8M**. Replacing the current generic coreless motors with these precision actuators will fully utilize the available power capacity and improve control loop linearity.
- **Custom Wheel Manufacturing:** The current wheels are modified commercial RC model tires, manually cut to reduce width. This manual process introduces slight imperfections and the rubber compound hardness is not ideal for the micromouse track. Future iterations should focus on manufacturing custom tires with optimized geometry and grip.

- **Weight Reduction:** Topological optimization can be applied to the 3D printed mechanical parts, introducing relief cuts to reduce mass and inertia without compromising structural rigidity.

6.1.2.4 Computation and Software

- **DFS Algorithm Optimization:** While the Depth-First Search with Manhattan heuristic pruning performs well in most scenarios, specific maze configurations caused the algorithm to stall during simulation. Further investigation and optimization are needed to handle these edge cases and guarantee convergence within the time limits.
- **Telemetry Client Application:** A Bluetooth communication protocol was developed for variable monitoring and synchronization. However, the client-side application (Frontend) remains in a prototype stage. Finalizing this software tool is crucial for real-time data visualization and efficient calibration of the system.

Bibliography

ARTFULBYTES. *Bots2d: 2D Robot Simulator*. [S.l.]: GitHub, 2025. <<https://github.com/artfulbytes/bots2d>>. Accessed: 2025-12-09. Citado na página 31.

BARBOSA, G. C.; PEREIRA, M. R. *Desenvolvimento de um robô de alto desempenho para a categoria Micro Mouse*. 2024. Trabalho de Conclusão de Curso (Engenharia de Computação) – Escola Politécnica da Universidade de São Paulo (USP). Disponível em: <https://pcs.usp.br/pcspf/wp-content/uploads/sites/8/2024/12/Monografia_PCS3860_COOP_2024_Grupo_C10.pdf>. Citado na página 17.

BARBOSA, G. C.; SANTI, P. H. M.; PEREIRA, M. R. *Micras Firmware - NTF Classic Micromouse project with a STM32 microcontroller*. 2025. GitHub Repository. Disponível em: <<https://github.com/Team-Micras/MicrasFirmware>>. Acesso em: 9 Dec 2025. Citado na página 55.

BORENSTEIN, J.; EVERETT, H. R.; FENG, L. *Where am I? Systems and methods for mobile robot positioning*. [S.l.], 1996. Section: Odometry and Dead Reckoning. Citado 2 vezes nas páginas 23 and 24.

CO., L. J. C. E. T. *CJA1117B-3.3 - 1A Low Dropout Linear Regulator*. [S.l.], 2025. Disponível em: <<https://item.szlcsc.com/datasheet/CJA1117B-3.3/24993.html>>. Acesso em: 9 Dec 2025. Citado na página 46.

DOUGLASS, B. *Design Patterns for Embedded Systems in C: An Embedded Software Engineering Toolkit*. Newnes/Elsevier, 2011. ISBN 9781856177078. Disponível em: <<https://books.google.com.br/books?id=8cgNIQEACAAJ>>. Citado 2 vezes nas páginas 31 and 55.

GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. [S.l.]: Addison-Wesley, 1994. Citado na página 56.

LAW, G. Quantitative comparison of flood fill and modified flood fill algorithms. *International Journal of Computer Theory and Engineering*, v. 5, n. 3, p. 503–508, June 2013. Disponível em: <<http://www.ijcte.org/papers/738-T100.pdf>>. Citado na página 62.

NTF. *Micromouse Contest Rules*. 2025. Nihon Technology Foundation. Disponível em: <https://www.ntf.or.jp/?page_id=933>. Acesso em: 9 Dec 2025. Citado 2 vezes nas páginas 21 and 22.

OGATA, K. *Modern Control Engineering*. 5th. ed. [S.l.]: Prentice Hall, 2010. Citado na página 24.

OSRAM. *SFH4550 - Infrared Emitting Diode*. [S.l.], 2025. Disponível em: <<https://www.alldatasheet.com/datasheet-pdf/view/218330/OSRAM/SFH4550.html>>. Acesso em: 9 Dec 2025. Citado na página 46.

OSRAM, A. *AS5047U Magnetic Position Sensor Datasheet*. [S.l.], 2025. Disponível em: <<https://look.ams-osram.com/m/48d90c982e0879e8/original/AS5047U-DS000637.pdf>>. Acesso em: 9 Dec 2025. Citado na página 47.

PETERSON, Z. *What is the Electrical Return Path in a PCB?* 2019. Updated: March 29, 2021. Accessed: 2025-12-09. Disponível em: <<https://resources.altium.com/p/what-return-current-path-pcb>>. Citado na página 53.

PETERSON, Z. *How to Design and Fabricate the Best 4 Layer PCB Stackup with Altium Designer*. 2021. Updated: July 16, 2024. Accessed: 2025-12-09. Disponível em: <<https://resources.altium.com/p/4-layer-pcb-stackup>>. Citado na página 52.

RUSSELL, S. J.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 3rd. ed. [S.l.]: Prentice Hall, 2010. Citado 2 vezes nas páginas 29 and 63.

SANTI, P. H. M. *Micras Firmware - NTF Classic Micromouse project with a STM32 microcontroller*. 2025. GitHub Repository. Disponível em: <<https://github.com/Team-Micras/MicrasHardware>>. Acesso em: 9 Dec 2025. Citado 2 vezes nas páginas 44 and 55.

SEMICONDUCTOR, A. . O. *AO3400A - N-Channel Enhancement Mode Field Effect Transistor*. [S.l.], 2025. Disponível em: <<https://item.szlcsc.com/datasheet/AO3400A/21629.html>>. Acesso em: 9 Dec 2025. Citado na página 46.

SIEGWART, R.; NOURBAKHSI, I. R.; SCARAMUZZA, D. *Introduction to Autonomous Mobile Robots*. 2nd. ed. [S.l.]: MIT press, 2011. Citado na página 23.

SINOPOWER. *HSBB3072 - N-Channel Enhancement Mode Field Effect Transistor*. [S.l.], 2025. Disponível em: <<https://item.szlcsc.com/datasheet/HSBB3072/3159665.html>>. Acesso em: 9 Dec 2025. Citado na página 45.

STMICROELECTRONICS. *L6205 - DMOS Dual Full Bridge for Motor Control Applications*. [S.l.], 2025. Disponível em: <<https://www.st.com/en/motor-drivers/l6205.html>>. Acesso em: 9 Dec 2025. Citado na página 49.

STMICROELECTRONICS. *L6981 - Synchronous Monolithic Step-Down Regulator*. [S.l.], 2025. Disponível em: <<https://www.st.com/en/power-management/l6981.html>>. Acesso em: 9 Dec 2025. Citado na página 46.

STMICROELECTRONICS. *LSM6DSV - 6-axis Inertial Measurement Unit (IMU) Datasheet*. [S.l.], 2025. Disponível em: <<https://www.st.com/resource/en/datasheet/lsm6dsv.pdf>>. Acesso em: 9 Dec 2025. Citado na página 49.

STMICROELECTRONICS. *stm32h725rg - High-Performance Arm Cortex-M7 Microcontroller*. [S.l.], 2025. Disponível em: <<https://www.st.com/en/microcontrollers-microprocessors/stm32h725rg.html>>. Acesso em: 9 Dec 2025. Citado na página 44.

STMICROELECTRONICS. *STSPIN958 - Compact 5 A brushed DC motor driver*. [S.l.], 2025. Disponível em: <<https://www.st.com/en/motor-drivers/stspin958.html>>. Acesso em: 9 Dec 2025. Citado na página 50.

STMICROELECTRONICS. *TSC2020 - High-precision, bidirectional, low-side current sense amplifier*. [S.l.], 2025. Disponível em: <<https://www.st.com/resource/en/datasheet/tsc2020.pdf>>. Acesso em: 9 Dec 2025. Citado na página 48.

TECHNOLOGY, H. *HM-19 - Bluetooth 5.0 BLE Module Datasheet*. [S.l.], 2025. Disponível em: <<https://wds-service-1258344699.file.myqcloud.com/20/12636/pdf/16705683758540c4d48696b84866a.pdf>>. Acesso em: 9 Dec 2025. Citado na página 51.

TI. *TPS61178 - Boost Converter*. [S.l.], 2025. Disponível em: <<https://www.ti.com/product/TPS61178>>. Acesso em: 9 Dec 2025. Citado na página 46.

TODOROV, E.; EREZ, T.; TASSA, Y. Mujoco: A physics engine for model-based control. In: IEEE. *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. [S.l.], 2012. p. 5026–5033. Citado na página 31.

TOSHIBA. *TPS601A - High Efficiency Step-Up DC-DC Converter Datasheet*. [S.l.], 2025. Disponível em: <<https://www.alldatasheet.com/datasheet-pdf/view/115616/TOSHIBA/TPS601A.html>>. Acesso em: 9 Dec 2025. Citado na página 46.